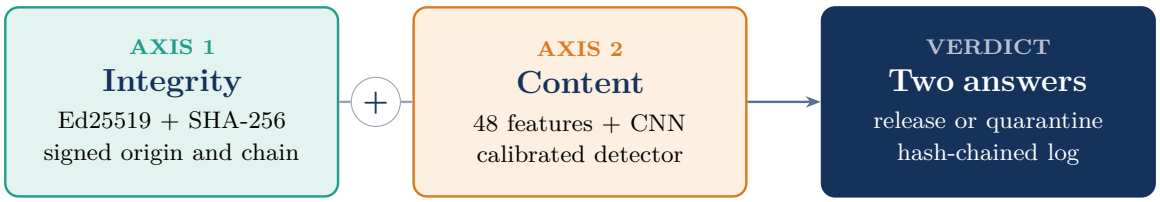
TSYP14 Technical Challenge · IEEE · Ground Truth Under Attack

Track 3: Trust the Pixel

PixelProof

Proving provenance and detecting post-capture manipulation of Earth-observation imagery

A cryptographic trust layer and a calibrated forensic detector, with two independent answers kept separate: was the pixel signed, and does the content look manipulated?



Phase 1 Report

IEEE Computational Intelligence Society Tunisia Chapter

IEEE GRSS Geoscience and Remote Sensing Society — IEEE Tunisia Section Chapter

IEEE Signal Processing Society TUNISIA CHAPTER

Contents

1 Executive summary **1**
1.1 Headline numbers . . . 1
1.2 Where to find each rubric item . . . 3

2 Scope definition **3**
2.1 Why this segment matters . . . 3
2.2 The real difficulty: two questions, not one . . . 3
2.3 Boundaries . . . 3

3 Threat model and segment analysis **4**
3.1 Assets and trust boundaries . . . 4
3.2 Attackers and controls . . . 4
3.3 STRIDE on the imagery segment . . . 4
3.4 Operator access: an extra boundary . . . 4

4 Objectives **6**

5 Data and manipulation-simulation plan **7**
5.1 Data sources . . . 7
5.2 Splits and leakage rules . . . 7
5.3 Manipulation generators: seen, held-out and frozen . . . 7
5.4 What is real and what is emulated . . . 8

6 System architecture **8**
6.1 Roles and deployment . . . 8
6.2 One downlink, end to end . . . 10
6.3 The ground station's code, module by module . . . 10
6.4 Technology choices . . . 10

7 The trust layer: digest, signed claim, chain and anchor **10**
7.1 Digest . . . 12
7.2 Signed claim and processing chain . . . 12
7.3 Trust anchor and registry . . . 12
7.4 Verification order . . . 12
7.5 Two axes, one verdict . . . 13
7.6 Audit log . . . 14

8 Signal and image processing **14**

9 AI / statistical detection and evaluation 16
9.1 Design . . . 16
9.2 Evaluation protocol . . . 17
9.3 Results on clean test patches . . . 18
9.4 Ablation . . . 18

10 Standard scenario, held-out test, evasion and response 19
10.1 Standard scenario and held-out variants . . . 19
10.2 Our own red team: adversarial iterations . . . 20
10.3 Robustness and the re-encoding gate . . . 20
10.4 Trust-layer attack campaign . . . 22
10.5 Attack scenarios A–F, run end to end . . . 22
10.6 Response . . . 22

11 Attack simulation and lab architectures 24
11.1 The programs and the attacker's modes . . . 24
11.2 Lab W: Windows host + Kali Linux VM (explicit relay, run live) . . . 25
11.3 Lab D1: Docker explicit relay . . . 26
11.4 Lab D2: Docker transparent man-in-the-middle . . . 26
11.5 Network namespaces and the three-VM VirtualBox lab . . . 27
11.6 Which lab proves what . . . 27

12 The ground-station dashboard 27
12.1 Pages and data sources . . . 28
12.2 Operator sign-in . . . 28

13 Feasibility, affordability and local execution 32

14 Business model and potential partners 32
14.1 Why the need is real in Tunisia . . . 32
14.2 Business Model Canvas . . . 33
14.3 Potential partners . . . 33
14.4 Partnership roadmap . . . 34

15 Testing, documentation and reproducibility 35
15.1 Testing and quality assurance . . . 35
15.2 Documentation, data and reproducibility . . . 35
15.3 Phase 1 deliverables and Phase 2 next steps . . . 36

16 Phase 2 work in progress: a candidate detector, measured 36
16.1 Why, and what changed . . . 37

16.2 Held-out v3, frozen before the candidate was trained 37
16.3 Results (fused models) 37
16.4 The cost, and how to read the gain 37

17 Assumptions and limitations **38**
17.1 Assumptions and technical risks 38
17.2 Limitations, with numbers 38

Abbreviations **39**

A How to run it **41**

B Integrity statuses and verdicts **41**

C API reference **41**

D Repository map **41**

E Data card (summary) **43**

References **43**

List of Figures

1 System overview: two independent axes, a cryptographic trust layer and a forensic content detector, combined into one verdict and logged. . . . 1

2 Our nine objectives grouped by what they protect. . . . 6

3 Data pipeline: from the official release to scored results; each step is a script in `scripts/`. . . . 7

4 Deployment view: the satellite signs, the untrusted channel may attack, the ground station verifies, analyses, decides and logs; the operator uses the dashboard, optionally behind a sign-in server. . . . 9

5 Sequence of one downlink through the relay. . . . 10

6 The provenance chain: each record is signed and points to the previous one; the received pixels must hash to the last signed digest. . . . 12

7 `Verifier.verify()`: the checks run in this order and the first failure decides the status. Live downlinks also record the product and its sequence number; analyst uploads only inspect. . . . 13

8 Authentic patch, the same patch after each attack family, the true mask, and the evidence heatmap for each. . . . 16

9 Content analysis: two models, one calibrated decision. . . . 16

10 PixelNet: the fixed high-pass front end makes the network read sensor noise instead of scenery; global max pooling keeps strong local evidence from a small edited region. 17

11 Confusion matrices of the fused model, seen and held-out variants, with thresholds for the false-alarm budget on validation authentics. . . . 18

12 Detection rate by manipulation family and test condition (seen and held-out variants, with and without laundering). . . . 20

13 Effect of one operation applied to every patch (an asterisk marks an operation never seen in training); the dashed line is the false-alarm budget. . . . 21

14 The dashboard's Attack lab running scenario F1 (manipulated before signing): every fake carries a valid signature and the content detector flags only some of them. The panel shows what the attacker did, the expected and actual result, the untouched control, the evidence digests and the timeline of the satellite, the attacker and the ground station. . . . 23

15 Lab W: satellite and ground station on the Windows host, the attacker in a Kali Linux VM on a VirtualBox host-only network. . . . 25

16 Lab D1: three containers on two networks; only the attacker is on both, so every message crosses it. . . . 26

17 Lab D2: the attacker is the satellite's router and silently redirects ground-bound traffic with one iptables rule; neither bridge has an internet route. . . . 26

18 Lab V: three VirtualBox VMs on internal networks, provisioned by Vagrant. Provided and syntax-checked, never booted by us. . . . 27

19 Overview: the headline figures are read from the result files, with the 3-D satellite → attacker → ground globe and the navigation. No number is typed by hand. . . 28

20 Analyse: a copy-move patch without provenance. Integrity says `NO_PROVENANCE`, the detector says copy-move (score far above the threshold), the verdict is `SUSPICIOUS` and quarantined, and the heatmap marks both copies. . . . 29

21 The Provenance page for a forged processing step: the claim and the two genuine steps are green, step 3 is where the chain breaks, and the received pixels are marked "not checked". (Redrawn from the page; the page itself is built from the verifier's reply.) 29

22 The sign-in screen: e-mail and password only; access is restricted to authorised operators. 30

23 Login flow: the tokens only ever travel in cookies that scripts cannot read; later calls (`/me`, `/auth/refresh`, `/logout`) carry the cookies automatically. 30

24 The responsive layout and the cookie flags, captured in the browser's developer tools. 31

25 The guided product tour: each step opens a page and spotlights its real panel. . . 31

26 Business Model Canvas of PixelProof. All entries are hypotheses to validate with partners. 33

List of Tables

1 Headline numbers of the shipped detector and trust layer. . . . 2
2 Rubric traceability (Phase 1: 100 points, weighted 30% of the final score). . . . 2
3 The three things an analyst needs to know, and which component answers each. . 3
4 Scope boundaries. . . . 4
5 Attacker profiles. . . . 5
6 STRIDE mapping. . . . 5
7 Project objectives, targets and how they are measured. . . . 6
8 Data sources. No private or restricted data is used; no paid service is needed. . . . 7
9 Manipulation methods per evaluation set. . . . 8
10 The roles and the programs that play them. . . . 9
11 The ground station's modules. . . . 11
12 Technology choices. . . . 11
13 The two-axis verdict table. . . . 13
14 The six forensic feature families (48 features in total). . . . 15
15 Evaluation metrics and how each is computed. . . . 17
16 Results on clean test patches (the fused model uses the shipped weights). . . . 18
17 False-alarm rate by land cover (seen test). . . . 18
18 Ablation of the feature model. . . . 19
19 Frozen held-out v2: the same set scored by the model before and after our changes. The false-alarm rate stays within the budget, so the detection gain is not bought with more false alarms. . . . 19
20 Adversarial iterations (development history; the iteration percentages are typed from the development log). . . . 20
21 Robustness sweep, shipped detector. "Seen in training" means the operation was part of the laundering used in training. . . . 21
22 Trust-layer campaign: every attack against every product. . . . 22
23 Attack scenarios A–F run end to end. . . . 23
24 The attacker's modes. . . . 24
25 Later Lab W sessions as recorded by the ground station. . . . 25
26 Which lab proves what. . . . 27
27 Dashboard pages and where each reads its data. . . . 28
28 Properties of the operator sign-in, read from the backend code. . . . 30
29 Cost of one decision: the challenge asks for latency, memory and security overhead. 32
30 Potential partners. Status of every row: not yet approached. . . . 33
31 The four test layers. . . . 35
32 Make targets. `make` does not exist on Windows; every target is one plain `python ...` line that can be run by hand. . . . 36
33 Phase 1 deliverables and Phase 2 next steps. . . . 36

34 What the candidate changes. Every choice was made on validation data. A wider network (width 32) was planned but not trained. 37

35 Shipped versus candidate detector on four test sets. The shipped columns of the first three rows come from the result files; the candidate columns and the v3 row are typed. 37

36 Detection by family, shipped → candidate. 38

37 The candidate's price (shipped column from the result files, candidate column typed). 38

38 Assumptions and technical risks. 39

40 Integrity statuses. 41

41 Verdicts and the response to each. 41

42 Ground-station API. Limits: 2 MB per image, decode guard, per-client rate limit on the analysis endpoints (`PIXELPROOF_RATE_PER_MIN`, default 120 per minute), no stack traces or key material in responses (tested). 42

43 Repository map. 42

1 Executive summary

The problem. An analyst receives a satellite image patch and must know two different things: was it really produced by the stated source and left untouched since (*integrity*), and does its content show signs of manipulation (*semantic truth*). These are separate questions, because a perfectly signed image can still have been faked before signing. PixelProof answers them on **two independent axes** and keeps both answers.

Our answer, in three points.

1. **Integrity is proven, not guessed.** Each patch is bound at the source to a signed claim (an Ed25519 signature over a SHA-256 digest of the pixels, plus product id, source, sensor and capture date) and to a signed, linked record of every processing step. The ground station trusts only registered public keys and pinned algorithms, remembers what it received (replay protection) and logs every decision in a hash chain.
2. **Content is assessed, honestly.** 48 explainable forensic features feed a gradient-boosted classifier, fused with a 74,116-parameter forensic CNN. The alarm threshold is calibrated on clean EuroSAT patches for a 5% [TARGET] false-alarm budget. The output is "manipulated (type)" or "no traces found", never "authentic".
3. **The verdict keeps both answers** (`VERIFIED`, `SIGNED_BUT_SUSPICIOUS`, `ALTERED_AFTER_CAPTURE`, `SUSPICIOUS`, `UNVERIFIED_NO_TRACES`), quarantines anything doubtful, and writes every decision to a hash-chained audit log.

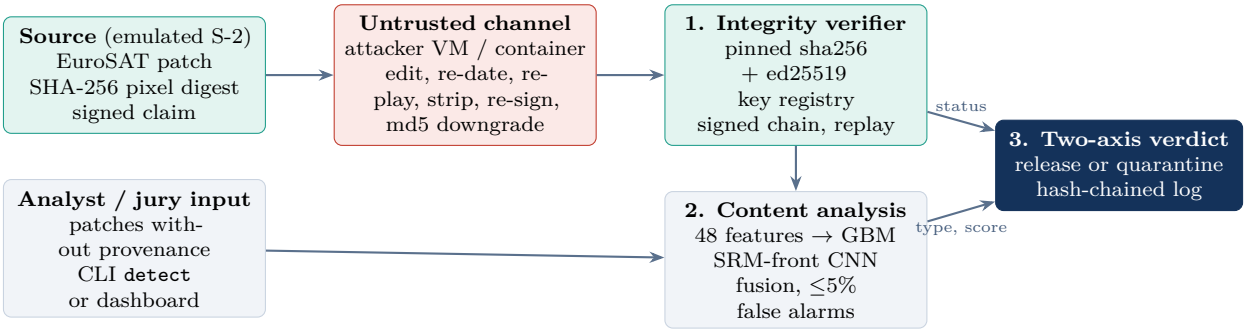


Figure 1: System overview: two independent axes, a cryptographic trust layer and a forensic content detector, combined into one verdict and logged.

1.1 Headline numbers

Shipped fused model; EuroSAT RGB; thresholds fixed on validation, never on test.

Phase 2 work already measured (Section 16). A retrained candidate detector, *not shipped*, raises held-out v1 detection to 75.2% and frozen held-out v2 to 84.6% at similar false-alarm rates, at the cost of more false alarms on re-encoded images. It is reported separately; every headline number above stays the shipped model's.

Main limitation. Smoothing of authentic patches (Gaussian blur, JPEG 2000) still raises false alarms: we do not yet tell benign smoothing from a resampling attack, so such a patch is quarantined for an analyst's review rather than released (Sections 10.3 and 17).

What we measured	Result	Source
Manipulation variants **seen** in training	**90.3%** detected at **3.6%** false alarms (AUC 0.972)	`metrics.json`
Held-out variants never seen in training	**48.2%** at 4.4% false alarms (AUC 0.793)	`metrics.json`
Frozen held-out v2 (methods fixed before any change)	**42.9%** at 4.3% false alarms (AUC 0.855)	`heldout_v2_v2model.json`
Trust layer: 11 attack types + genuine, 200 signed products each	every attack caught with its expected status (Section 10.4)	`integrity_campaign.json`
Live lab: Windows host + Kali VM relay	11/11 cryptographic attacks stopped, 8/8 genuine unharmed (Section 11.2)	Section 11.2
Transparent man-in-the-middle (Docker)	16/16 stopped; the satellite's own capture never shows the attacker (Section 11)	`docker_mitm/`
Cost (one CPU core)	36.7 ms per patch; sign 0.25 ms, verify 0.38 ms	`metrics.json`, `integrity_campaign.json`

Table 1: Headline numbers of the shipped detector and trust layer.

Rubric criterion (points)	Sections	Evidence in the repository
Threat model & relevance (15)	3	`docs/THREAT_MODEL.md`, `docs/SECURITY_ANALYSIS.md`
Architecture & trust mechanism (15)	6, 7, 11, 12	`pixelproof/provenance.py`, `ledger.py`, `service.py`, `results/integrity_campaign.json`
Signal / image processing (15)	8	`pixelproof/features.py`, `tests/test_content.py`
Detector design & evaluation (20)	5, 9, 16	`pixelproof/detector.py`, `cnn.py`, `results/metrics.json`
Standard scenario & evasion (20)	10, 11	`results/heldout_v2_v2model.json`, `results/robustness.json`, labs in `results/`
Feasibility, affordability, local execution (10)	13, 14	`results/metrics.json`, `Dockerfile`, `requirements.txt`
Clarity & reproducibility (5)	15, 15.2	`README.md`, `docs/RUN_GUIDE.md`, `Makefile`, `scripts/check_numbers.py`

Table 2: Rubric traceability (Phase 1: 100 points, weighted 30% of the final score).

1.2 Where to find each rubric item

Check our claims yourself in five minutes.

(1) `python -m pytest -q` runs the automated tests.

(2) `python scripts/check_numbers.py docs/REPORT.md` proves every number in the Markdown report resolves to a result file; `python scripts/render_latex.py --check` does the same for this document.

(3) `python scripts/quick_demo.py` runs satellite → attacker → ground on one laptop and opens the dashboard.

2 Scope definition

Our scope in one sentence. The imagery segment of the Earth-observation chain: from the moment a Sentinel-2 product leaves its trusted source to the moment an analyst uses a 64×64 EuroSAT patch derived from it. For every patch, PixelProof answers two questions before anything is released: *who signed it, and are the pixels unchanged?* and *does the content show traces of manipulation?*

2.1 Why this segment matters

Earth-observation products drive decisions in agriculture, insurance, water management and disaster response. A failure in this chain can be silent: the pipeline keeps running while a corrupted observation reaches the analyst. Radio-link attacks (Track 1) and sensor-level false-data injection (Track 2) are out of scope, which is our design choice, consistent with the track statement.

2.2 The real difficulty: two questions, not one

Question	What answers it	What it cannot say
Provenance: who signed this claim, and when?	Ed25519 signature checked against a trusted public-key registry	Whether the picture is true
Integrity: are these exactly the pixels that were signed?	SHA-256 digest over the pixels and a signed processing chain	Whether the content was faked before signing
Visual authenticity: does the picture show what really happened?	Statistical content detector (features + CNN)	Anything with certainty: the output is "no traces found", never "authentic"

Table 3: The three things an analyst needs to know, and which component answers each.

2.3 Boundaries

	Details
In scope	Signed origin and processing chain; replay and downgrade protection; hash-chained audit log; forensic detection of resampling/splicing, copy-move and generated/inpainted regions; operator dashboard with optional sign-in; attack labs
Out of scope	Radio-link attacks (Track 1); sensor-level false-data injection (Track 2); volumetric denial of service; fully generated GAN or diffusion patches (none exist in the official data)
Users	Analyst (verdict, release of quarantined data), ground-station operator (dashboard, keys), jury (CLI `detect` on held-out patches)
Data	EuroSAT RGB (Sentinel-2, 27,000 patches of 64×64, 10 land-cover classes), MIT licence
Scaling	One ground station verifies every product of one source; additional sources only need a registered public key

Table 4: Scope boundaries.

3 Threat model and segment analysis

> **Rubric: threat model (15).** We deliver a segment and trust-boundary analysis, six attacker profiles, a STRIDE mapping [14], and the AI component that enables the attack. Evidence: `docs/THREAT_MODEL.md`.

3.1 Assets and trust boundaries

Assets. (1) the pixel values; (2) the capture claim (product id, source, sensor, capture date, bands); (3) the processing history; (4) trust material: the source and processor private keys and the public-key registry; (5) the detector (model file and thresholds) and the audit log.

Trust boundaries.

- **TB1 source:** the private key never leaves it.
- **TB2 transport:** anyone may read, modify, replay or drop messages (the attacker of our labs sits here).
- **TB3 ground processing:** only registered processors may add signed steps.
- **TB4 storage and API:** stored files may be swapped; the ground station re-verifies and logs.

3.2 Attackers and controls

The AI component that facilitates the attack. Generative inpainting and synthesis (diffusion, GANs, learned inpainting) let an attacker erase or invent content with plausible texture at almost no cost [7], and learned post-processing erases forensic traces. AI also powers our detector, as in recent work on satellite-image manipulation [6], but Section 10 shows how much any learned detector degrades on manipulations it has never seen, which is the main reason provenance comes first.

3.3 STRIDE on the imagery segment

3.4 Operator access: an extra boundary

The dashboard is itself an attack surface. An optional sign-in server gates access with JWT tokens [13] stored in `HttpOnly` cookies, so that page scripts cannot read them (Section 12.2). This protects

ID	Attacker	Capability	Our control
A1	network man-in-the-middle	modify, replay, strip or swap manifests	signature, pixel digest, key registry, sequence numbers
A2	storage insider	edit pixels or claims, add fake processing	digest re-verification, signed chain, audit log
A3	AI-equipped forger (unsigned imagery)	resample, splice, copy-move, inpaint, re-encode	content detector, 5% false-alarm budget
A4	adaptive attacker who knows the detector	feathering, colour matching, rotations, laundering	held-out evaluation, laundering-robust training, provenance first
A5	key thief or compromised payload	signs arbitrary content	revocation; content analysis still runs on signed data
A6	algorithm substitution	claims md5 or none and recomputes it	algorithm pinning (sha256 + ed25519 only)

Table 5: Attacker profiles.

Threat	Example	Control → status	Residual risk
Spoofing	re-sign a re-dated patch with an own key	trusted registry → `UNKNOWN_KEY`	stolen key until revoked
Tampering	inpainting, copy-move, resampling, LSB after capture	pixel digest → `PIXELS_ALTERED`	unsigned imagery relies on statistics
Repudiation	a processor denies an alteration	signed steps and hash-chained log	—
Information disclosure	LSB embedding as a covert channel	any bit change breaks the digest	invisible to content analysis
Denial of service	flooding with large files	2 MB limit, decode guard, rate limit, quarantine instead of crash	volumetric DoS out of scope
Elevation of privilege	forged processing step to legitimise an edit	step signature and link → `CHAIN_BROKEN`	compromised processor key

Table 6: STRIDE mapping.

the *dashboard*; the ground-station ingest API has no authentication or TLS in the isolated lab, which is listed as a limitation (Section 17).

4 Objectives

Main objective. Prove where every pixel came from, assess what it shows without overclaiming, never silently release a doubtful patch, keep the analyst in charge of releasing quarantined data, and make every decision auditable, on a laptop with no GPU.

The challenge asks for reported detection quality, false alarms, latency, memory and security overhead; it sets no numeric targets. The 5% false-alarm budget is ours and is fixed on validation; the other targets are stated qualitatively and measured on frozen sets.

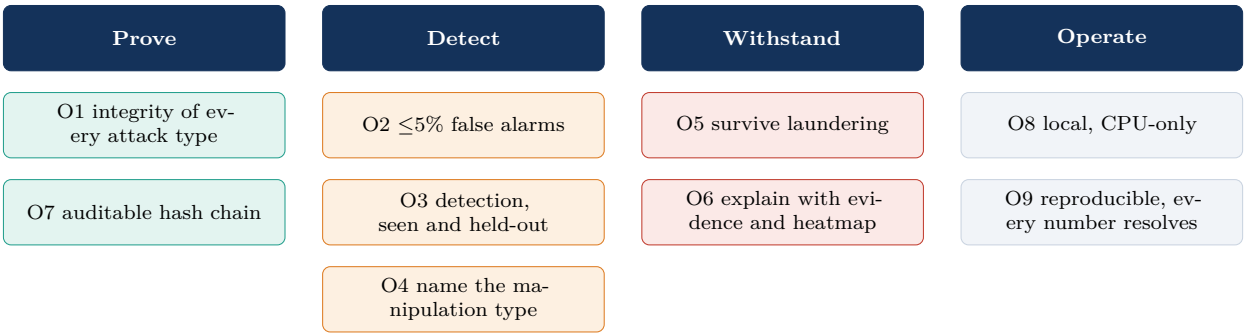


Figure 2: Our nine objectives grouped by what they protect.

#	Objective	Target	How we measure it
O1	Prove integrity	every cryptographic attack returns its expected status; genuine products 100% [TARGET]	trust-layer campaign (200 products × 11 attacks), live labs
O2	Keep false alarms low	≤5% on clean authentic validation patches [TARGET]	threshold fixed on validation; false-alarm rate per land cover
O3	Detect manipulation	high detection on seen variants, honest detection on held-out variants	seen, held-out v1, frozen v2 (and v3 for the candidate)
O4	Name the manipulation type	4-class macro-F1 reported on seen and held-out	confusion matrices
O5	Survive laundering	report false alarms and detection under re-encoding and smoothing	robustness sweep
O6	Explain	plain-words evidence and a heatmap on every alarm	feature z-scores; heatmap IoU
O7	Be auditable	100% of decisions in a hash chain; tampering detected	ledger verification on every refresh
O8	Run locally	CPU only, one core, laptop memory	latency probe, peak memory
O9	Be reproducible	one command per step; every number resolves	`render_latex.py`, `check_numbers.py`, `make reproduce`

Table 7: Project objectives, targets and how they are measured.

5 Data and manipulation-simulation plan

In one sentence. Authentic public Sentinel-2 patches (EuroSAT) are split without leakage; every manipulation is generated by deterministic, seeded code, so for every patch we know whether it was altered, how, and where, and the methods used for testing are kept apart from those used for training.

5.1 Data sources

Source	What we take	Used for
EuroSAT RGB [1]	27,000 Sentinel-2 patches of 64×64 pixels, 10 land-cover classes, MIT licence; fetched by `scripts/download_eurosat.py`, which refuses data whose class counts differ	authentic patches; every split and every number in this report
`samples/authentic/`	30 unmodified official patches shipped in the repository	labs, tests and the demo run without the 2 GB dataset
Emulated source and processor	signing keys `sat-01` and `proc-01`, capture claims, signed processing steps (software keystore)	provenance of the 11-attack campaign and the labs
Jury patches	held-out authentic and manipulated patches supplied by the organisers (any format or size)	the standard scenario; read by `pixelproof. cli detect`

Table 8: Data sources. No private or restricted data is used; no paid service is needed.

5.2 Splits and leakage rules

Every official EuroSAT patch is assigned to exactly one split by a SHA-256 hash of its id: *train* 60%, *validation* 15%, *test-seen* 12.5% and *test-held-out* 12.5%. For each source patch we render one authentic and one manipulated copy. A patch, its manipulated copy and its splicing donor always share a split, so there is **no leakage**. Thresholds and the fusion weight are fixed on validation only. The seen test set holds 6680 patches and the held-out test set 6612. The full data card is `docs/DATASET_CARD.md`.

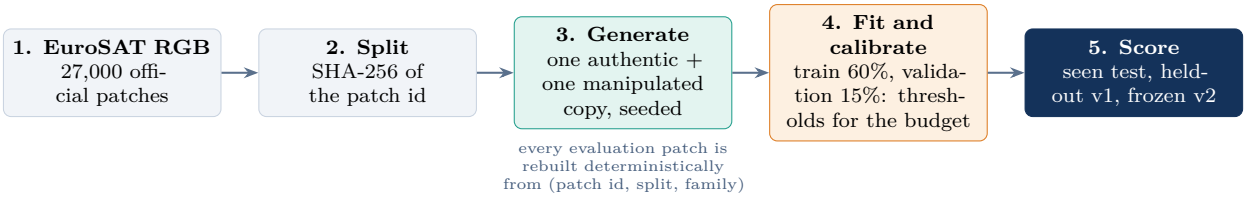


Figure 3: Data pipeline: from the official release to scored results; each step is a script in `scripts/`.

5.3 Manipulation generators: seen, held-out and frozen

The standard scenario asks us to detect a resampled or spliced image, a copy-move alteration and a generated or inpainted region. Our generators cover these three groups with methods that are **different on purpose** between training and testing, so the held-out numbers measure generalisation, not memorisation.

Freezing protocol. Held-out v2 was generated and its SHA-256 recorded before any detector

Group	Training (seen)	Held-out v1	Held-out v2 (frozen before any change)
Resampled / spliced	bilinear ×1.15–1.5, rotation 4–12°, hard-edged splices	bicubic/Lanczos, ×1.6–2.2, rotation 15–35°, feathered colour-matched splices	perspective/biquadratic warp; Poisson (seamless-clone) splices, donor from another land-cover class
Copy-move	translated rectangles	rotated/rescaled/noisy polygons	mirrored elliptical region, soft-blended
Generated / inpainted	Telea inpainting	Navier–Stokes, biharmonic, texture synthesis	exemplar fill + injected sensor noise
Laundering (all patches)	1–2 of JPEG/WebP/noise	JPEG 2000 + radiometric stretch	—

Table 9: Manipulation methods per evaluation set.

change made after v1:

```
f29c4beec8b08f6708abcd08b13d8b594c5af7d6498b3ecf698f5d0a4e5e945b
```

The v1-model baseline on it is stored next to the result of the shipped model, so the gain from our changes is honest. A third set, held-out v3 (thin-strip regions; Fourier anisotropic resampling, Laplacian-pyramid strip splicing [9], multi-clone strip copy-move, push-pull multiscale fill with grain), was generated from the seen-test split with methods that appear nowhere else and frozen before the Phase 2 candidate was trained. It holds 6680 patches (3340 authentic, 532 resampled, 552 spliced, 1090 copy-move, 1166 inpainted); its SHA-256 is

```
6384021722ac848b4b97874f9d1b4ec61015e268b105ec9311910e4c27f33d29
```

The scoring script refuses to score the same model on v3 twice.

5.4 What is real and what is emulated

- **Real:** the pixels (official Sentinel-2 patches), the cryptography (Ed25519, SHA-256 from a standard library), the network attacks (real packets through a real relay or router), the detector.
- **Emulated:** the satellite and its processing chain (a Python process holding two software keys), the capture claim fields, and the manipulations of unsigned imagery (generated by our own code, not by third-party generative models).

6 System architecture

> **Rubric: architecture & trust mechanism (15).** This chapter shows how the parts fit; Section 7 details the trust mechanism. Evidence: the repository layout (Appendix D).

6.1 Roles and deployment

PixelProof reproduces the three roles of the challenge as separate programs that talk over HTTP, so they can run on one laptop, in three containers, or on separate (virtual) machines without code changes.

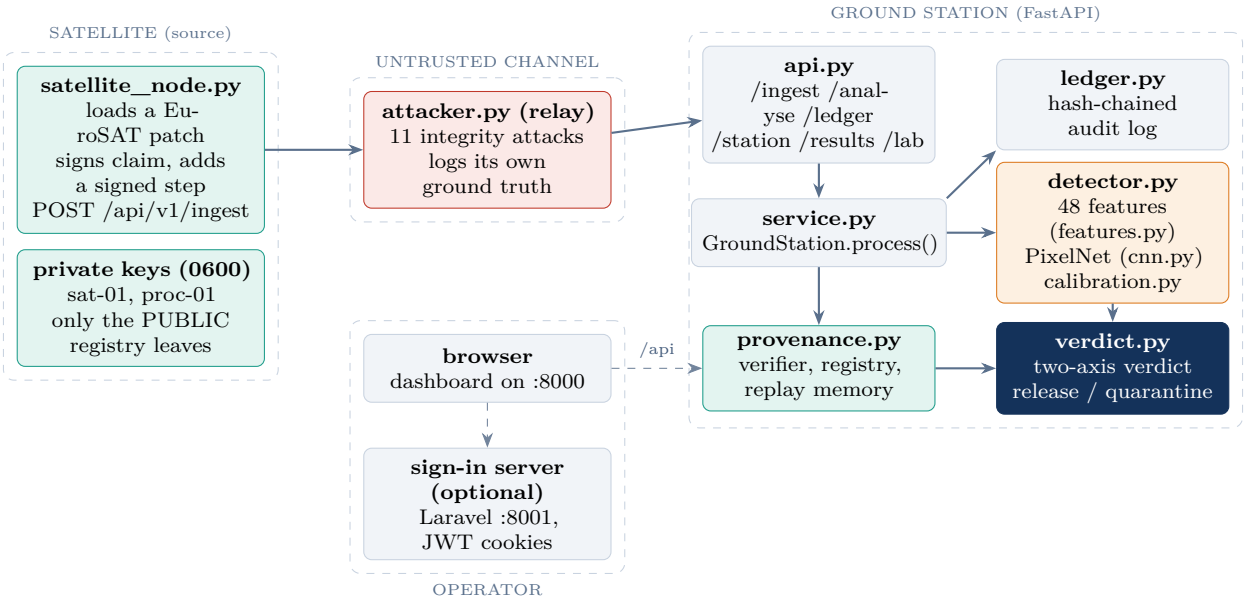


Figure 4: Deployment view: the satellite signs, the untrusted channel may attack, the ground station verifies, analyses, decides and logs; the operator uses the dashboard, optionally behind a sign-in server.

Role	Program	What it does
Satellite (source)	`simulation/satellite_node.py`	loads a EuroSAT patch, signs a capture claim, adds a signed processing step, posts it
Untrusted channel	`simulation/attacker.py`	an HTTP relay that applies one of 11 integrity attacks at a chosen rate and logs what it did
Ground station	`pixelproof/`, served by `uvicorn pixelproof.api:app`	verifies, runs the content detector, decides, logs, serves the API and the dashboard
Scoring	`simulation/score_demo.py`	compares the attacker's own log (ground truth) with the ground station's answers
Operator	browser + `pixelproof/static/app`	the dashboard; an optional Laravel sign-in server (`backend/`)

Table 10: The roles and the programs that play them.

6.2 One downlink, end to end

The message is a JSON object with an `image_png_b64` field and a `manifest` field, posted to `POST /api/v1/ingest`. The PNG is lossless, so the pixels the ground station decodes are exactly the pixels the satellite hashed unless someone changed them. The ground station decodes with a 2 MB limit and a decode guard, then calls `GroundStation.process(img, manifest, live=True)`, which verifies, analyses, combines and appends to the audit log, and returns the verdict. The audit-log entry is written before the answer leaves the ground station.

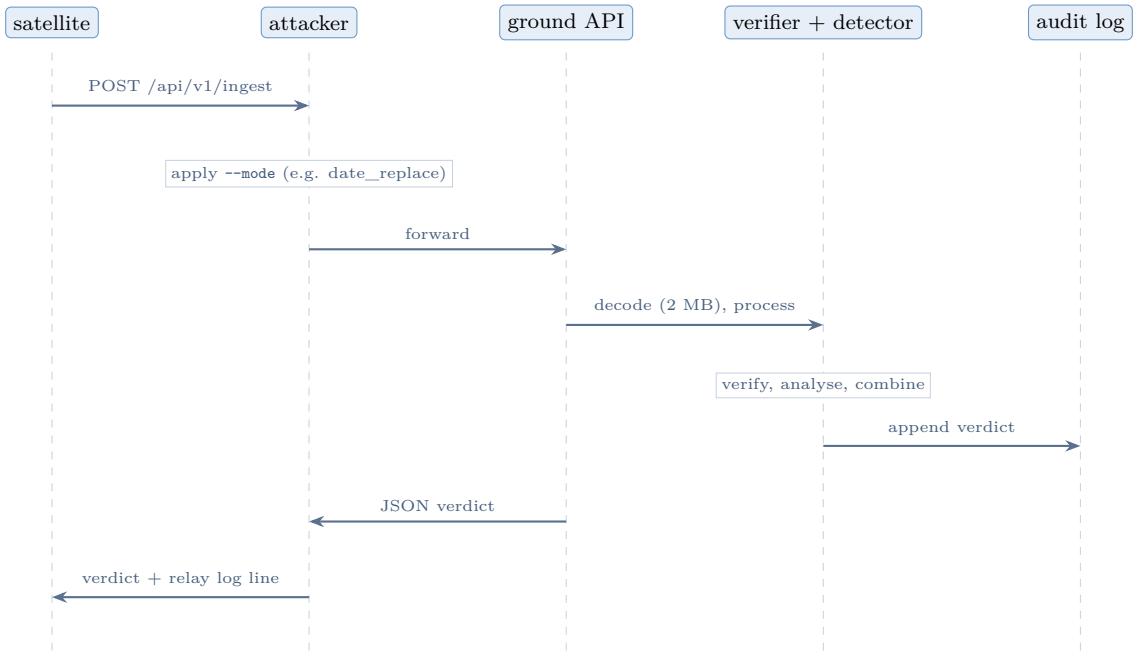


Figure 5: Sequence of one downlink through the relay.

6.3 The ground station's code, module by module

The orchestration is deliberately short; this is the whole decision path (`service.py`, abridged):

```
def process(self, img, manifest, live, source="upload"):
    with self._lock:
        if live:
            self.reload_registry()
        integrity = self.verifier.verify(img, manifest, record=live)  # proof
        content = self.detector.analyse(img)                           # evidence
        result = combine(integrity, content)                           # two-axis verdict
        result["pixel_digest"] = pixel_digest(img)
        entry = self.ledger.append({"channel": source, "verdict": result["verdict"],
                                    "integrity": integrity.status, ...})  # hash chain
        result["ledger_hash"] = entry["hash"]
        return result
```

6.4 Technology choices

7 The trust layer: digest, signed claim, chain and anchor

> **Rubric: architecture & trust anchor (15).** We deliver a signed digest + claim + processing chain, a software key registry with revocation and algorithm pinning, replay protection and a hash-chained audit log. Evidence: `pixelproof/provenance.py`, `pixelproof/ledger.py`,

Module (`pixelproof/`)	Responsibility	Key entry points
`provenance.py`	the trust layer: digest, keystore, registry, signers, verifier	`pixel_digest`, `SourceSigner.capture`, `ProcessorSigner.record`, `Verifier.verify`
`service.py`	orchestration of one decision; replay memory rebuilt at start	`GroundStation.process`, `summarize`, `event_severity`
`verdict.py`	the two-axis decision and the response	`combine`
`ledger.py`	append-only, hash-chained audit log	`Ledger.append`, `Ledger.verify`
`features.py`	48 forensic features and the evidence heatmap	`extract`, `heatmap`
`cnn.py`	PixelNet, training, test-time averaging, loading	`PixelNet`, `train`, `predict_proba`
`detector.py`	fusion, calibrated decision, explanation, persistence	`Detector.analyse`, `Detector.load`
`calibration.py`	the re-encoding gate and condition-aware threshold	`train_gate`, `adaptive`
`inputs.py`	any format and size; tiling without resizing	`load_any`, `analyse_any`
`manipulations.py`, `dataset.py`	deterministic manipulation generators and evaluation sets	`make`, `launder`, `jobs_for_split`
`heldout_v2.py`, `heldout_v3.py`	frozen held-out sets (never imported by training)	`make`
`integrity_attacks.py`, `lab.py`	the 11 network attacks; scenarios A–F	`ATTACKS`, `SCENARIOS`
`api.py`	FastAPI endpoints, limits, lab switch	Appendix C
`cli.py`	command line: `detect`, `analyse`, `sign`, `verify`, `keys-init`, `fingerprints`	`main`

Table 11: The ground station's modules.

Layer	Technology	Why
Ground station	Python 3.12, FastAPI, uvicorn	small, typed API; runs anywhere Python runs
Cryptography	`cryptography` (Ed25519 [11], SHA-256 [12])	standard, fast, deterministic signatures
Detection	scikit-learn, PyTorch (CPU), OpenCV, scikit-image	CPU-only, no GPU or cloud
Dashboard	React 18, TypeScript, Vite, Tailwind CSS, Three.js, Motion	offline build, no CDN, strict Content-Security-Policy
Optional sign-in	Laravel 12, JWT in `HttpOnly` cookies, MySQL	an access screen for a shared ground station; absent → open access
Labs	Docker Compose, VirtualBox (Kali), iptables, tcpdump	isolated, reproducible attack environments

Table 12: Technology choices.

> `make campaign`.

7.1 Digest

`SHA-256("PXP1" | dtype | shape | raw pixel bytes)` [12]. Re-saving a file without changing pixels keeps the digest; flipping one bit changes it. It is computed over pixels, not the file, so the same claim holds whether the patch travels as PNG, TIFF or lossless WebP.

```
def pixel_digest(img: np.ndarray) → str:
    a = np.ascontiguousarray(img)
    head = f"PXP1|{a.dtype.str}|{'x'.join(map(str, a.shape))}|".encode()
    return sha256_hex(head + a.tobytes())
```

7.2 Signed claim and processing chain

Signed claim. The claim holds `format`, `product_id`, `source`, `sensor`, `capture_time`, `bands`, `shape`, `dtype`, `pixel_digest`, `hash_alg`, `sig_alg`, `key_id`, `seq` and `prev_hash`. It is serialised as canonical JSON (sorted keys, no spaces) and signed with Ed25519 [11, 8] by the source key. `seq` and `prev_hash` chain the captures of one source (replay and re-order detection).

Processing chain. Each step appends `op`, `params`, `input_digest`, `output_digest`, `key_id` and `prev_record`, signed by a registered processor key. The verifier walks the chain: every step must be validly signed, start from the previous output, and link to the previous record; the final digest must equal the digest of the pixels received.

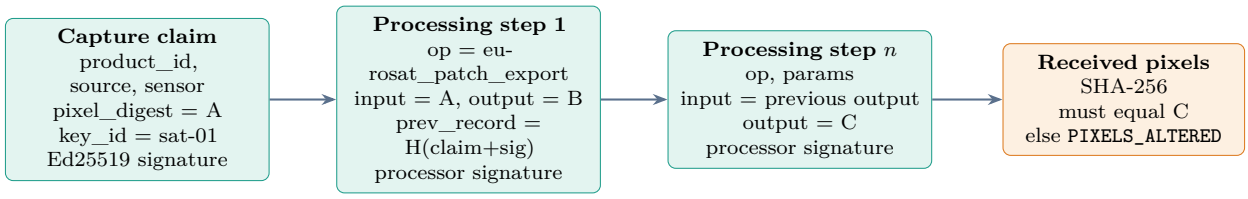


Figure 6: The provenance chain: each record is signed and points to the previous one; the received pixels must hash to the last signed digest.

```
claim = {"format": FORMAT, "product_id": product_id, "source": self.source, "sensor": self.sensor,
         "capture_time": capture_time, "bands": list(bands), "shape": list(img.shape),
         "dtype": str(img.dtype), "pixel_digest": pixel_digest(img), "hash_alg": "sha256",
         "sig_alg": "ed25519", "key_id": self.key_id, "seq": self.seq, "prev_hash": self.head}
manifest = {**claim, "signature": b64e(self._sk.sign(canonical(claim))), "processing": []}
```

7.3 Trust anchor and registry

A software keystore (Ed25519 private keys outside the repository, permissions 0600) is explicitly allowed by the rules. The ground station holds only a **registry of public keys** with roles (`source`, `processor`) and revocation flags; a key that arrives inside a message is never trusted. Out-of-band trust is a human comparing the SHA-256 fingerprints of the public keys on both machines (`python -m pixelproof.cli fingerprints`, and the dashboard's Station page). Upgrade path without code changes: the same signer interface backed by SoftHSM2 or a TPM through PKCS#11. The signed claim plus signed processing history follows the same idea as the C2PA Content Credentials [10], restricted to what a single ground station can verify offline.

7.4 Verification order

The order matters: algorithms are pinned **before** any digest is compared (a downgraded md5 claim never reaches the comparison), the claim signature is checked before the processing chain, and the

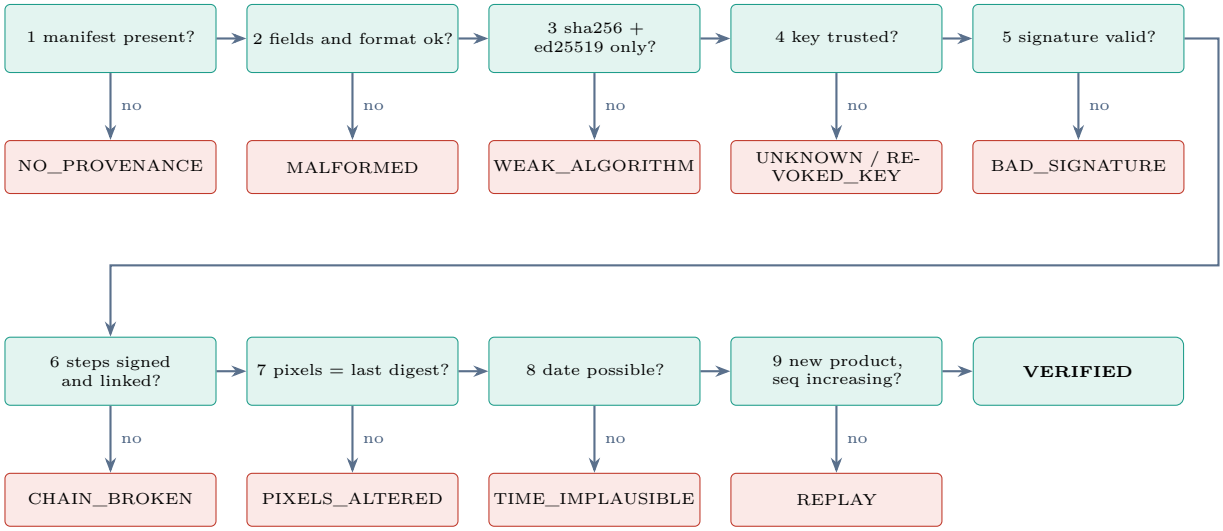


Figure 7: `Verifier.verify()`: the checks run in this order and the first failure decides the status. Live downlinks also record the product and its sequence number; analyst uploads only inspect.

pixels are compared with the **last** signed digest. Live downlinks (`/ingest`) also record the product and the source's sequence number, so a second delivery is a `REPLAY`; this memory is **rebuilt from the audit log when the ground station restarts** (tested), so a reboot does not reopen the replay window. Analyst uploads (`/analyse`) only inspect.

7.5 Two axes, one verdict

	content: no traces found	**content: manipulation found**
integrity VERIFIED	`VERIFIED`	`SIGNED_BUT_SUSPICIOUS` (*faked before signing?*)
integrity FAILED	`ALTERED_AFTER_CAPTURE`	`ALTERED_AFTER_CAPTURE` (+ type, heatmap)
no provenance	`UNVERIFIED_NO_TRACES`	`SUSPICIOUS` (+ type, heatmap)

Table 13: The two-axis verdict table.

```python
def combine(integrity, content):
    manipulated = bool(content and content.get("manipulated"))
    if integrity.status == "VERIFIED":
        verdict = "SIGNED_BUT_SUSPICIOUS" if manipulated else "VERIFIED"
    elif integrity.status == "NO_PROVENANCE":
        verdict = "SUSPICIOUS" if manipulated else "UNVERIFIED_NO_TRACES"
    else:
        verdict = "ALTERED_AFTER_CAPTURE"
    action = "RELEASE" if verdict in {"VERIFIED", "UNVERIFIED_NO_TRACES"} else "QUARANTINE"
```

What a signature proves, and what it does not. *Provenance*: who signed this claim, and when (the key is in our registry). *Integrity*: are these exactly the pixels that were signed. *Visual authenticity*: does the picture show what really happened. A signature answers the first two and says nothing about the third, because a compromised payload can sign a fake. In our campaign, of 200 fakes signed before capture the content detector flagged 168 and 32 passed as `VERIFIED`. The cost on the other side: of 200 genuine signed products, 9 were flagged by the content detector and quarantined for review although their integrity was proven. That is why the two axes are reported separately and read together.

7.6 Audit log

Every decision is appended to a JSON-lines file in which each entry carries the hash of the previous one (`ledger.py`). The dashboard re-verifies the whole chain on every refresh; editing or deleting any past line turns the badge to "BROKEN" and names the first bad entry (tested). The log is *tamper-evident, not tamper-proof*: a writer who rewrites every line consistently could rebuild a valid chain, because nothing outside anchors it (Section 17).

```
def append(self, event):
    prev = self._head()
    body = {"time": now_utc_iso(), "prev": prev, **event}
    entry = {**body, "hash": sha256_hex(canonical(body))}
    self.path.open("a").write(json.dumps(entry, sort_keys=True) + "\n")
```

Measured overhead (200 products, one CPU core): signing takes 0.25 ms and verification 0.38 ms (median). The signed manifest with one processing record averages 1091 bytes, which is 17.2% of the size of the 64×64 PNG it protects (the overhead is fixed, so it shrinks for larger images).

8 Signal and image processing

> **Rubric: signal/image processing (15).** We deliver six explainable forensic feature families with the physics of each trace, and an evidence heatmap. Evidence: `pixelproof/features.py`, `tests/test_content.py`.

All features are computed on the 64×64 patch (luminance $g = 0.299R + 0.587G + 0.114B$ unless stated). Images of any other size are cut into 64×64 tiles and **never resized**, because resizing would itself add resampling traces.

The SRM high-pass residual uses the kernel

$$r = g * \tfrac{1}{4}\begin{bmatrix} -1 & 2 & -1 \\ 2 & -4 & 2 \\ -1 & 2 & -1 \end{bmatrix}, \qquad p = \exp\Big(-\frac{e^2}{2\hat{\sigma}^2}\Big), \quad \hat{\sigma} = 1.4826\,\mathrm{MAD}(e),$$

where $e = g - k * g$ is the residual of the 4-neighbour linear predictor used by the Popescu–Farid map. To catch rotated or mirrored copies, each 8×8 block's key is the lexicographically smallest of its eight D4 variants; the eight DCT variants are derived algebraically from a single transform, so the matcher costs one DCT per block.

The resampling peaks are *periodic*, which is why they are informative for resampling; they feed the classifier, but a separate decision between periodic resampling and aperiodic smoothing is **not implemented yet** (Sections 10.3 and 17).

Evidence heatmap. The maximum of the noise-inconsistency map, the over-smooth map and the copy-move mask, smoothed: it shows *where* the evidence is. Each alarm also lists, in plain words, the features furthest (more than 2.5 standard deviations) from the distribution of authentic training patches. Localisation is the weakest part: mean IoU against the true mask is 0.31 for copy-move, 0.11 for splicing and 0.02 for inpainting. Classification is reliable; precise location is Phase 2.

Family	Physics of the trace	What we compute
Resampling `rs_*` [2]	Interpolation makes each pixel a fixed linear combination of its neighbours, creating *periodic* correlations	(a) mean absolute second derivative of each column and row as a 1-D signal; Fourier peak over median at the resampling period. (b) Popescu–Farid probability map of the residual of a 4-neighbour linear predictor; peak ratio and radius of its 2-D spectrum, $\log \hat{\sigma}$, kurtosis
Frequency profile `fq_*`	Inpainting and generation remove high frequencies; local edits disturb the JPEG grid	Hann-windowed power spectrum in 8 radial bands, log–log slope, high/low energy ratio, strength of the 8×8 JPEG block grid (authentic EuroSAT RGB is single-JPEG)
Noise residual `nz_*` [4]	Real sensor noise is spatially consistent; edited regions are too smooth or too noisy	SRM high-pass residual, local standard deviation in 8×8 blocks, robust z-scores: level, spread, range, extremes per block and per 16×16 neighbourhood, share of outlier and "too smooth" blocks, kurtosis
Channel coherence `cc_*`	Sensor and demosaicing noise is correlated across R, G, B; pasted or synthesised regions break that	correlation of the three residuals, globally and in 16×16 windows
Colour statistics `cs_*`	Spliced regions carry colour or noise that does not match the host, even with a feathered seam	local z-scores of chroma (Cb, Cr) and saturation, share of outlying regions, local noise-level inconsistency
Copy-move `cm_*` [3]	Duplicated regions share low-frequency content, possibly rotated or mirrored	overlapping 8×8 blocks matched on 9 low-frequency DCT coefficients; keys canonicalised over the 8 dihedral (D4) transforms; matches grouped by a common symmetry and translation; textureless blocks (water, cloud, bare soil) skipped

Table 14: The six forensic feature families (48 features in total).

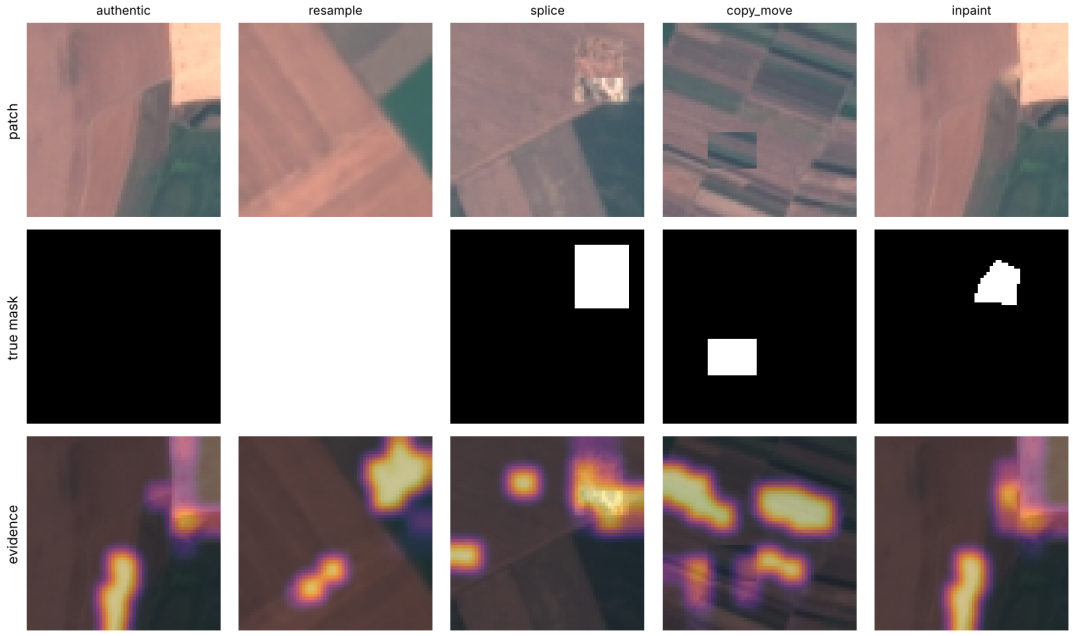


Figure 8: Authentic patch, the same patch after each attack family, the true mask, and the evidence heatmap for each.

9 AI / statistical detection and evaluation

> **Rubric: detector design & evaluation (20).** We deliver a fused feature+CNN detector, a calibrated false-alarm budget with a re-encoding gate, leakage-free splits (Section 5), an ablation, and bootstrap confidence intervals. Evidence: `pixelproof/detector.py`, `scripts/evaluate.py`, `make evaluate`.

9.1 Design

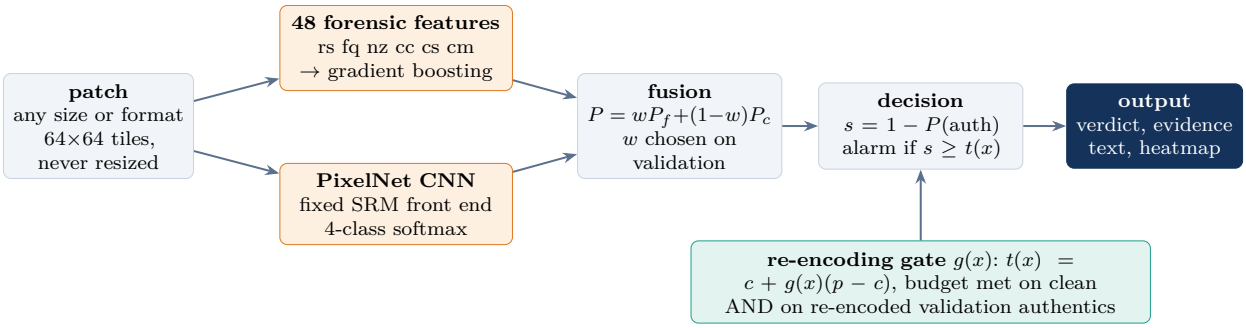


Figure 9: Content analysis: two models, one calibrated decision.

Feature model. Scikit-learn `HistGradientBoostingClassifier` on the 48 features, four classes: authentic, resampled/spliced, copy-move, generated/inpainted.

Pixel model (PixelNet). A 74,116-parameter forensic CNN, in the spirit of constrained and SRM-initialised forensic networks [5, 4]. A *fixed* front end applies three SRM high-pass kernels per colour channel; residuals are truncated to ±3 and concatenated with the centred RGB patch (12 channels). Then three stages of two 3×3 convolutions (16/32/64 channels, batch norm, ReLU), global average and max pooling, and a linear layer. Trained with AdamW and a one-cycle schedule, using only flips and 90° rotations (they preserve forensic traces).

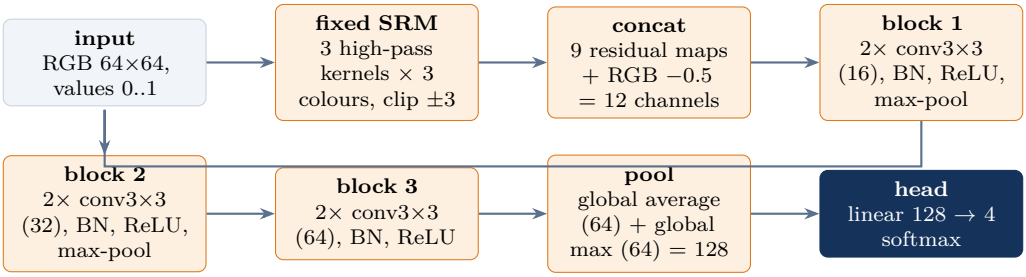


Figure 10: PixelNet: the fixed high-pass front end makes the network read sensor noise instead of scenery; global max pooling keeps strong local evidence from a small edited region.

```python
def forward(self, x):                       # x: (N, 3, 64, 64) in [0, 1]
    r = F.conv2d(x * 255.0, self.srm, padding=2, groups=3)  # 9 fixed SRM residual maps
    r = torch.clamp(r, -3, 3) / 3.0         # truncation (TLU)
    h = torch.cat([r, x - 0.5], 1)          # + centred RGB = 12 channels
    h = F.max_pool2d(self.b1(h), 2)
    h = F.max_pool2d(self.b2(h), 2)
    h = self.b3(h)
    h = torch.cat([h.mean((2, 3)), h.amax((2, 3))], 1)       # global average + max
    return self.head(self.drop(h))
```

Fusion and decision. $P = w\,P_{\text{features}} + (1-w)\,P_{\text{CNN}}$ with $w = 0.4$ chosen on validation. The manipulation score is $s = 1 - P(\text{authentic})$. The alarm threshold is condition-aware: $t(x) = c + g(x)\,(p - c)$, where $g(x)$ is the probability the patch was re-encoded after capture (a second gradient-boosting model, validation accuracy 96.8% [MEASURED]), $c = 0.436$ gives the 5% budget on clean validation authentics and $p = 0.638$ gives it on re-encoded ones (Section 10.3 explains why).

9.2 Evaluation protocol

Table 15 lists what we measure and how. Thresholds are never tuned on a test set, and every interval is a 95% bootstrap interval over test patches.

Metric	Method
Detection rate at a fixed false-alarm budget	threshold set on clean validation authentics; reported on test, never tuned on test
AUC-ROC, AUC-PR	score $s = 1 - P(\text{authentic})$ over authentic and manipulated test patches
False-alarm rate	authentic test patches, overall and per land cover; 95% bootstrap intervals
Four-class macro-F1 and confusion matrix	authentic / resampled-spliced / copy-move / generated-inpainted, on detected and undetected patches
Generalisation	seen variants versus held-out v1, frozen held-out v2 (SHA-256 recorded), and v3 for the candidate
Robustness	one operation (JPEG, WebP, blur, noise, JPEG 2000, gamma) applied to every patch; false alarms and detection reported
Trust layer	200 signed products × 11 attacks, scenarios A–F, three lab architectures; every status compared with its expected value
Efficiency	latency (p50, p95), throughput, peak memory, model size, signing and verification time, manifest size
Explanation	heatmap IoU against the true mask; evidence text from the features furthest from authentic

Table 15: Evaluation metrics and how each is computed.

9.3 Results on clean test patches

Model	Test set	AUC-ROC	Detection	False alarms	4-class macro-F1
Features (GBM)	seen	0.844	60.9%	4.1%	0.705
Pixel CNN	seen	0.959	84.9%	4.1%	0.852
Fused	**seen**	**0.972**	**90.3%**	**3.6%**	**0.903**
Features (GBM)	held-out	0.687	29.5%	4.7%	0.398
Pixel CNN	held-out	0.780	46.1%	3.9%	0.522
Fused	**held-out**	**0.793**	**48.2%**	**4.4%**	**0.528**

Table 16: Results on clean test patches (the fused model uses the shipped weights).

95% bootstrap intervals (fused): seen detection [89.3%, 91.1%], seen false alarms [2.9%, 4.1%], held-out detection [46.7%, 49.9%].

Takeaway: fusing the two models beats either alone on seen variants at a lower false-alarm rate; every detector loses ground on manipulations it has never seen, which Section 10 quantifies.

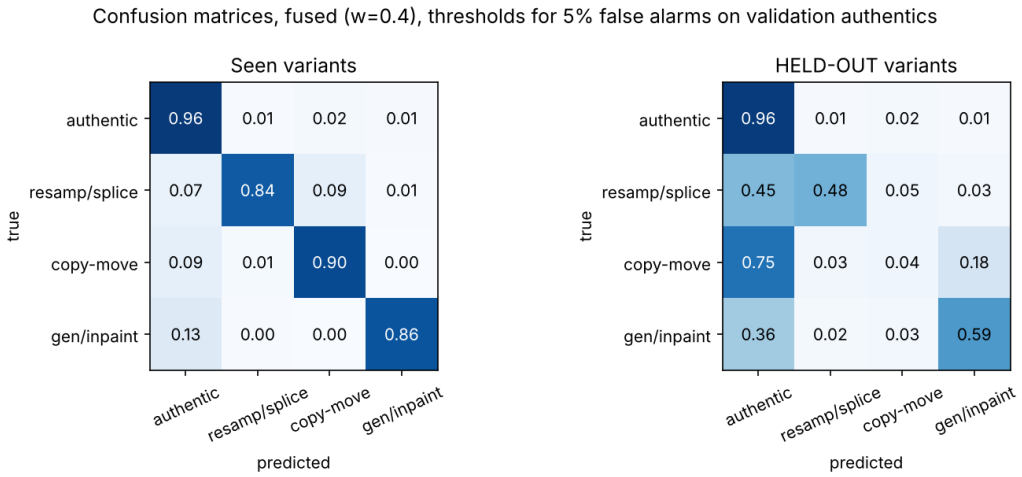


Figure 11: Confusion matrices of the fused model, seen and held-out variants, with thresholds for the false-alarm budget on validation authentics.

False alarms by land cover (seen test): every class stays near the 5% budget; the highest is Industrial at 8.0%.

	AnnualCrop	Forest	Herbaceous	Highway	Industrial	Pasture	PermanentCrop	Residential	River	SeaLake
false alarms	3.2%	4.1%	1.3%	2.5%	8.0%	1.3%	3.8%	4.7%	2.3%	4.0%

Table 17: False-alarm rate by land cover (seen test).

9.4 Ablation

We retrain the feature model without one feature group at a time (AUC-ROC; the CNN is not part of this ablation). The largest drop on seen variants is the copy-move group (0.844 to 0.751); on held-out variants it is channel coherence (0.687 to 0.652). The ablation does *not* show a held-out benefit for every group: removing the frequency group leaves held-out AUC slightly higher, and

Feature set	AUC seen	AUC held-out
all 48 features	0.844	0.687
without resampling (`rs_*`)	0.840	0.687
without frequency (`fq_*`)	0.832	0.692
without noise residual (`nz_*`)	0.841	0.679
without channel coherence (`cc_*`)	0.826	0.652
without copy-move (`cm_*`)	0.751	0.683
without colour statistics (`cs_*`)	0.836	0.680
no laundering augmentation	0.845	0.688

Table 18: Ablation of the feature model.

removing the resampling group leaves it unchanged. We keep all six because the plain-language evidence of an alarm is drawn from all of them.

10 Standard scenario, held-out test, evasion and response

> **Rubric: standard scenario & evasion (20).** We deliver the standard scenario, two frozen held-out sets with unseen methods, a robustness sweep, the trust-layer attack campaign, scenarios A–F and the response. Evidence: `scripts/robustness_sweep.py`, `scripts/heldout_v2.py`, `scripts/run_attack_campaign.py`.

10.1 Standard scenario and held-out variants

Authenticate clean patches, then detect a resampled or spliced image, a copy-move alteration and a generated or inpainted region, with held-out authentic and manipulated patches supplied by the jury. Our rehearsal uses manipulation variants **never seen in training** (the methods are listed in Table 9 of Section 5). Held-out v2 was created and SHA-256-frozen before any detector change made after v1; its fingerprint and the v1-model baseline on it are recorded, so the gain from our changes is honest.

Frozen held-out v2 (fused)	v1 model (baseline)	v2 model (shipped)
detection rate	32.0%	**42.9%**
false-alarm rate	4.2%	**4.3%**
AUC-ROC	0.755	**0.855**
copy-move	9.8%	**41.7%**
splice	14.2%	**17.4%**
resample	59.6%	**67.5%**
inpaint	50.9%	**45.1%**

Table 19: Frozen held-out v2: the same set scored by the model before and after our changes. The false-alarm rate stays within the budget, so the detection gain is not bought with more false alarms.

Held-out v1, detection by family (fused): resampling 96.6%, inpainting/generation 63.8%, copy-move 25.3%, splicing 18.1%.

Interpretation. Detectors that read *physical* traces generalise: interpolation leaves periodic correlations whatever the kernel, and every inpainting method leaves a region without sensor noise. Detectors that rely on *shortcut* cues do not, which is where an adaptive attacker (A4) goes, and

why provenance carries the guarantee.

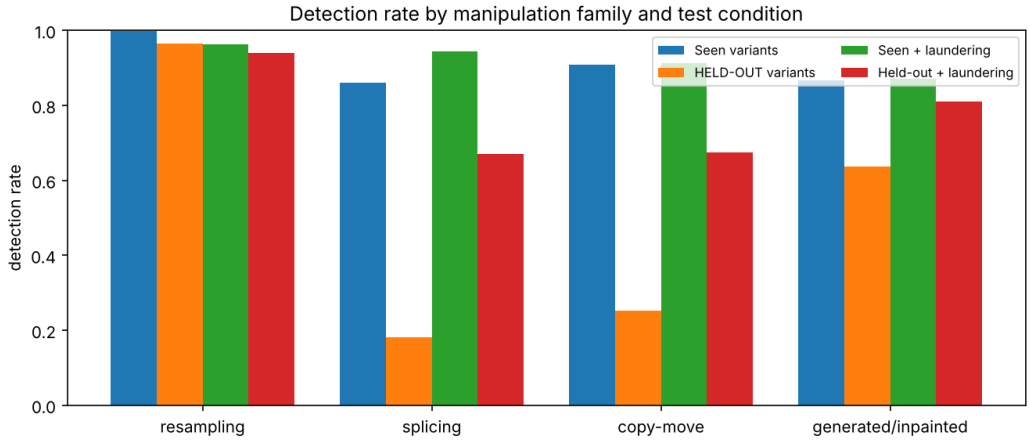


Figure 12: Detection rate by manipulation family and test condition (seen and held-out variants, with and without laundering).

10.2 Our own red team: adversarial iterations

Each iteration was evaluated on data never used for its development, and the measured failure drove the next design.

#	Problem found	Measured effect	Fix	Result
1	mild JPEG/noise augmentation only	14% false alarms on authentics after stronger re-encoding	widen augmentation	lower, but still leaking
2	unseen re-encodings (WebP, blur)	45% of authentics looked manipulated	calibrate on clean validation	fewer, see #3
3	blur treated as benign	held-out resampling collapsed 85% → 33% (blur *is* low-pass)	remove blur from the benign set	resampling restored
4	re-encoded authentics still flagged	29–66% false alarms at JPEG q90–q45	re-encoding gate and condition-aware threshold	see sweep below
5	translation-only copy-move, hard-edge-only splice	held-out copy-move and splice weak	D4-invariant matching, colour-statistics features, feathered-edge augmentation	held-out v2 table above

Table 20: Adversarial iterations (development history; the iteration percentages are typed from the development log).

10.3 Robustness and the re-encoding gate

A detector trained on EuroSAT learns "single-JPEG EuroSAT", so a re-saved authentic patch can look manipulated. The gate (validation accuracy 96.8%) moves the threshold between two calibrated points, keeping re-encoded authentics near the budget. The robustness sweep applies one operation to every one of 600 source patches:

Operation	Seen in training	False alarms	Detection
none	yes	3.5%	88.0%
JPEG q90	yes	5.5%	46.3%
JPEG q70	yes	3.8%	31.7%
JPEG q45	yes	4.3%	25.2%
JPEG q25	no	12.2%	23.0%
WebP q60	yes	7.2%	34.3%
blur 0.6	no	100.0%	100.0%
blur 1.2	no	100.0%	100.0%
noise 3	yes	0.5%	30.7%
noise 6	no	1.2%	20.0%
JPEG 2000 r10	no	69.3%	88.5%
JPEG 2000 r30	no	87.3%	87.8%
gamma 0.8	no	7.8%	87.8%
gamma 1.3	no	3.3%	85.2%

Table 21: Robustness sweep, shipped detector. "Seen in training" means the operation was part of the laundering used in training.

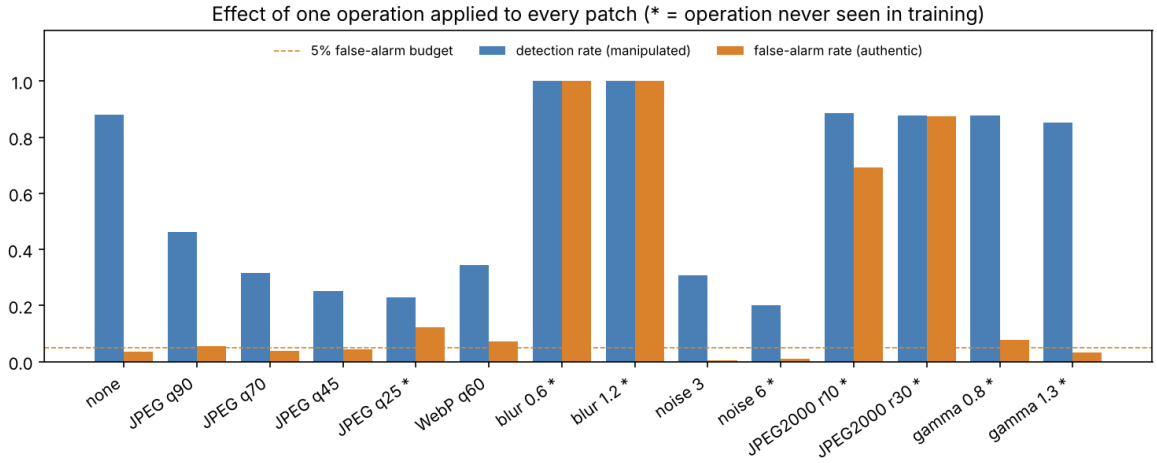


Figure 13: Effect of one operation applied to every patch (an asterisk marks an operation never seen in training); the dashed line is the false-alarm budget.

Honest consequence. Gaussian blur and JPEG 2000 on authentic patches still raise false alarms sharply, because they are physically the same low-pass signal as resampling. There is **no separate "globally processed" outcome yet**: a globally smoothed authentic patch is flagged like any manipulation (`SIGNED_BUT_SUSPICIOUS` with a valid signature, `SUSPICIOUS` without) and quarantined for an analyst. This costs a review, not a silently released fake. The integrity axis still proves, independently, that signed pixels are untouched since signing.

10.4 Trust-layer attack campaign

200 genuine signed products, each attacked in transit:

Attack	Expected status	Caught
none (genuine)	`VERIFIED`	100%
inpaint after capture	`PIXELS_ALTERED`	100%
copy-move after capture	`PIXELS_ALTERED`	100%
resample after capture	`PIXELS_ALTERED`	100%
LSB embedding (invisible)	`PIXELS_ALTERED`	100%
date replaced, signature kept	`BAD_SIGNATURE`	100%
date replaced and re-signed (attacker key)	`UNKNOWN_KEY`	100%
hash downgraded to md5	`WEAK_ALGORITHM`	100%
forged processing step	`CHAIN_BROKEN`	100%
manifest attached to another image	`PIXELS_ALTERED`	100%
replay of a delivered product	`REPLAY`	100%
edit + provenance stripped	`NO_PROVENANCE`	100% (see note)

Table 22: Trust-layer campaign: every attack against every product.

Note on the last row. "100%" means the verifier correctly reported that no provenance exists, **not** that the edit was stopped. Without a manifest only the content detector can notice: of 200 edited patches whose manifest was removed, 138 were flagged `SUSPICIOUS` and 62 were released as `UNVERIFIED_NO_TRACES`, i.e. not noticed.

10.5 Attack scenarios A–F, run end to end

`python -m simulation.run_attack --all --n 20` signs synthetic traffic from the bundled EuroSAT sample patches, attacks it, and feeds it to the real ground-station code (evidence for every trial in `results/attack_runs/`). Every trial first delivers an untouched control product, which must be `VERIFIED`, so a scenario can never pass because the ground station rejects everything. The first block is cryptographic: the expected status is a guarantee, and n is the number of trials per scenario. The visual block has no signature to check, so the result is a measured detection rate, on manipulation methods the detector was not trained on, and it is small-sample.

On the 20 untouched originals used as controls, the detector raised 0 false alarms in E1 and 0 in E4. Scenario F1 is the honest limit of the whole design: a valid signature does not make a fake true.

10.6 Response

Anything other than `VERIFIED` or `UNVERIFIED_NO_TRACES` is **quarantined**: kept, withheld downstream, released only by a logged analyst decision. The audit log is append-only and hash-chained; editing any past entry is detected (tested). The labs of Section 11 run the full satellite → attacker

ID	Scenario (what the attacker does)	Expected	Result
A1	image tampering: erase and fill a region, keep the manifest	`PIXELS_ALTERED`	20/20
A2	invisible tampering: flip 512 least-significant bits	`PIXELS_ALTERED`	20/20
B1	manifest tampering: change the capture date	`BAD_SIGNATURE`	20/20
B2	identity forgery: claim to be the satellite, sign with an own key	`UNKNOWN_KEY`	20/20
C1	edit the image and rebuild a matching manifest, signed with an own key	`BAD_SIGNATURE`	20/20
C2	algorithm substitution: claim md5	`WEAK_ALGORITHM`	20/20
D1	replay of a genuine, already delivered message	`REPLAY`	20/20
E1	copy-move, manifest removed	detector flags it	flagged 4/20
E2	splicing, manifest removed	detector flags it	flagged 2/20
E3	inpainting, manifest removed	detector flags it	flagged 9/20
E4	resampling, manifest removed	detector flags it	flagged 19/20
F1	fake made **before** signing (compromised payload)	integrity `VERIFIED`	`VERIFIED` 20/20; detector flagged 9/20

Table 23: Attack scenarios A–F run end to end.

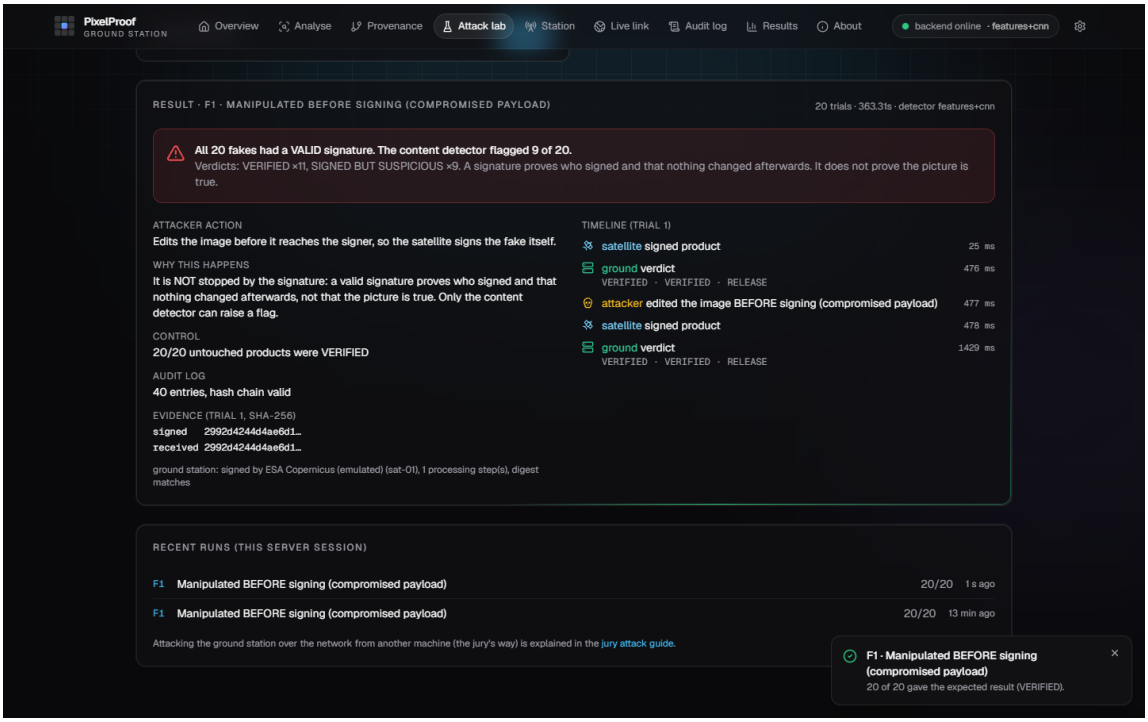


Figure 14: The dashboard's Attack lab running scenario F1 (manipulated before signing): every fake carries a valid signature and the content detector flags only some of them. The panel shows what the attacker did, the expected and actual result, the untouched control, the evidence digests and the timeline of the satellite, the attacker and the ground station.

→ ground chain and score it with `simulation/score_demo.py`.

11 Attack simulation and lab architectures

> **Rubric: standard scenario & evasion (20) and architecture (15).** We deliver the same three programs deployed four ways, from an explicit relay through a real Kali VM to a transparent man-in-the-middle whose transparency is proven by the satellite's own packet capture. Evidence: `results/docker_lab/`, `results/docker_mitm/`, `results/lab/`, `docker-compose*.yml`, `lab/`, `vm/README.md`.

11.1 The programs and the attacker's modes

`satellite_node.py` signs each patch with the source key (`sat-01`), adds one signed processing step (`proc-01`), and posts it to its configured target. It keeps its sequence counter on disk, so a second run is not a replay. `attacker.py` is a threaded HTTP server: for every message it decides (with probability `--rate`) whether to attack, applies the chosen attack from `pixelproof/integrity_attacks.py`, forwards the result to the ground station, and writes one line of ground truth (`attack`, `expected_integrity`, `ground_integrity`, `ground_verdict`).

```
choice = "passthrough"
if a.mode != "passthrough" and rng.random() < a.rate:
    choice = a.mode if a.mode != "random" else str(rng.choice(list(ATTACKS)))
if choice in ("replay", "manifest_swap") and not hist:      # needs an earlier message
    choice = "passthrough"
...
status, body = forward(a.ground.rstrip("/"), out)
rec = {"product_id": ..., "attack": choice, "expected_integrity": ATTACKS[choice][1] if choice in ATTACKS
       else "VERIFIED", "ground_verdict": res.get("verdict"), "ground_integrity": res["integrity"]["status"]}
```

Mode	What the relay does	Expected ground status
`pixel_inpaint`, `pixel_copy_move`, `pixel_resample`	edits the pixels, keeps the manifest	`PIXELS_ALTERED`
`lsb_embed`	flips 512 invisible lowest bits	`PIXELS_ALTERED`
`manifest_swap`	valid manifest on the previous genuine image	`PIXELS_ALTERED`
`date_replace`	capture date −365 days, signature kept	`BAD_SIGNATURE`
`date_replace_resign`	changes the date and re-signs with its own key	`UNKNOWN_KEY`
`algorithm_downgrade`	edits pixels, claims md5	`WEAK_ALGORITHM`
`forged_processing`	hides an edit behind a fake processing step	`CHAIN_BROKEN`
`replay`	re-sends an earlier genuine message	`REPLAY`
`strip_manifest`	edits and deletes the manifest	`NO_PROVENANCE` (the detector decides)

Table 24: The attacker's modes.

`score_demo.py` joins the relay's log with the ground station's answers and verifies the audit-log chain; a cryptographic attack counts as stopped only if the ground station returned exactly the

expected status.

11.2 Lab W: Windows host + Kali Linux VM (explicit relay, run live)

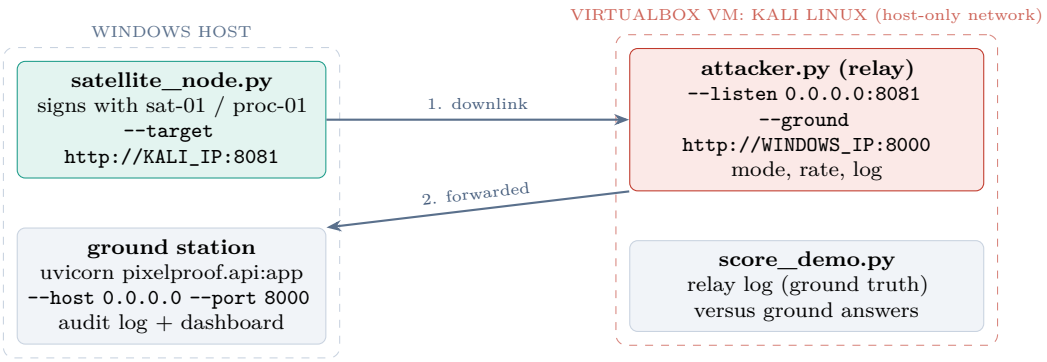


Figure 15: Lab W: satellite and ground station on the Windows host, the attacker in a Kali Linux VM on a VirtualBox host-only network.

The satellite and the ground station run directly on the Windows host; only the attacker runs in a VM (Kali Linux in VirtualBox). The satellite is pointed at the relay on purpose (`--target http://KALI_IP:8081`); the relay forwards to `http://WINDOWS_IP:8000`, so the ground station listens on `0.0.0.0`. This is an **explicit relay**, not a transparent attacker (that is Lab D2). The VM holds no private key: it can change messages but cannot sign as the satellite. One emulation shortcut: because both ends share the host, the satellite writes the public registry that the ground station reads; in a real deployment the fingerprints are compared out of band (Section 7).

```
# Kali VM
python3 simulation/attacker.py --listen 0.0.0.0:8081 --ground http://WINDOWS_IP:8000 \
    --mode date_replace --rate 1.0 --log runtime/attack_date_replace.jsonl

# Windows host
uvicorn pixelproof.api:app --host 0.0.0.0 --port 8000
python simulation\satellite_node.py --target http://KALI_IP:8081 --n 5 --interval 1.5
```

Source of the Lab W figures. The values below are typed in this document from the live session; the corresponding evidence folder is not among the result files this document is built from, so unlike every other number they are not regenerated by `render_latex.py`.

Results. The mixed run (`--mode random`, 20 messages) scored against the relay's own log: cryptographic attacks stopped with the expected status 11/11, genuine messages unharmed 8/8, 1 provenance-stripped edit decided by the content detector (released as `UNVERIFIED_NO_TRACES`), audit-log chain valid: True (237 entries). Later sessions recorded by the ground station: For these

Session	Messages	Ground station's integrity statuses	Reading
replay test	60	`REPLAY` 11, `VERIFIED` 49	every resent product refused
date replacement	40	`BAD_SIGNATURE` 20, `VERIFIED` 20	Ed25519 catches the re-dating
date replacement	60	`BAD_SIGNATURE` 40, `VERIFIED` 20	same
date replacement	40	`BAD_SIGNATURE` 20, `VERIFIED` 20	one genuine message `SIGNED_BUT_SUSPICIOUS` (1): a content false alarm

Table 25: Later Lab W sessions as recorded by the ground station.

later sessions the Kali-side logs were not copied, so the attack mode is inferred from the ground station's answers, not proven by the relay's log. `algorithm_downgrade`, `forged_processing` and `manifest_swap` were not run on this setup (they are covered by the campaign, the Docker labs and the Attack lab).

11.3 Lab D1: Docker explicit relay

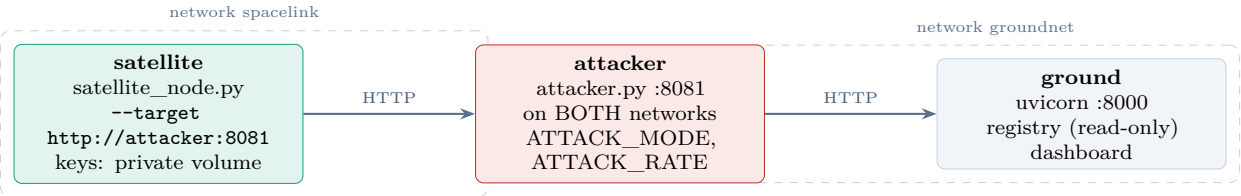


Figure 16: Lab D1: three containers on two networks; only the attacker is on both, so every message crosses it.

`docker compose up --build` builds one image and starts three services. The satellite is only on `spacelink`, the ground station only on `groundnet`, the attacker on both; private keys live in a volume mounted only by the satellite, and the ground station receives the public registry read-only. `ATTACK_MODE` and `ATTACK_RATE` select the attack.

Result (`results/docker_lab/`): 40 downlinks, 19 of 19 cryptographic attacks stopped with the expected status, genuine messages unharmed 20/20, 1 manifest-stripped edit (released as `UNVERIFIED_NO_TRACES`: 1, i.e. the content detector did **not** notice it), audit-log chain valid: True (40 entries).

11.4 Lab D2: Docker transparent man-in-the-middle

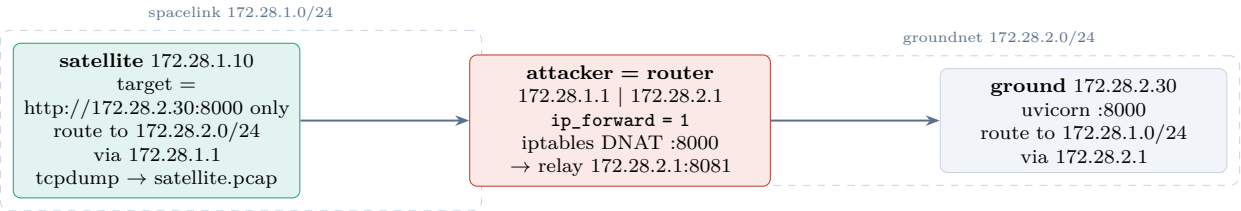


Figure 17: Lab D2: the attacker is the satellite's router and silently redirects ground-bound traffic with one iptables rule; neither bridge has an internet route.

The satellite is configured with **one** address, the ground station's (`http://172.28.2.30:8000`); it has never heard of the attacker. Its only route to the ground subnet goes through the attacker container, which forwards packets and adds a standard destination-NAT rule that hands ground-bound traffic to its relay:

```
attacker:
  cap_add: [NET_ADMIN]
  sysctls: { net.ipv4.ip_forward: "1" }
  command: |
    runuser -u app -- python simulation/attacker.py --listen 172.28.2.1:8081 \
        --ground http://172.28.2.30:8000 --mode $${ATTACK_MODE} --rate $${ATTACK_RATE} ... &
    iptables -t nat -A PREROUTING -p tcp -d 172.28.2.30 --dport 8000 \
        -j DNAT --to-destination 172.28.2.1:8081
```

No ARP spoofing and no packet forging are involved; the reply is rewritten back, so it appears to come from the ground station. Both bridges have address translation switched off (no internet route), and `NET_ADMIN` applies only inside each container's own network namespace. **Proof of transparency:** the satellite records its own traffic with `tcpdump`; `scripts/check_transparency.py` reads that

capture.

Result (`results/docker_mitm/`): the satellite's capture holds 466 IP packets, all with the ground station as peer (attacker address in any IP header: False); 16 of 16 cryptographic attacks stopped, genuine messages unharmed 7/7, audit-log chain valid: True.

11.5 Network namespaces and the three-VM VirtualBox lab

`lab/netns_lab.py` builds the same routed topology with Linux network namespaces (no containers, root needed); its recorded run caught tampering 7/7 (`results/lab/`) and it has not been re-run on the Windows laptop.

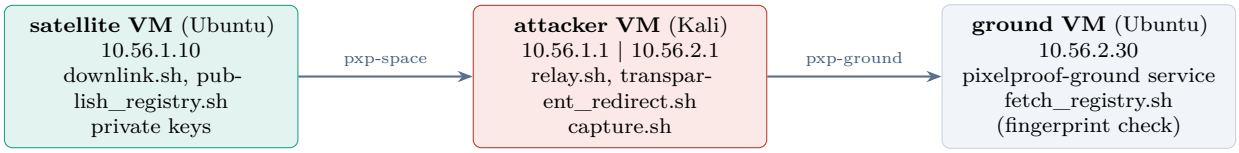


Figure 18: Lab V: three VirtualBox VMs on internal networks, provisioned by Vagrant. Provided and syntax-checked, never booted by us.

`lab/Vagrantfile` provisions three VMs (Ubuntu satellite and ground, Kali attacker) on VirtualBox internal networks with the same addresses as the routed design (`10.56.1.0/24`, `10.56.2.0/24`), a systemd ground service with `NoNewPrivileges` and a read-only system, and helper scripts for the out-of-band fingerprint check, the relay, the transparent redirect and packet capture. We did not boot it (no Vagrant on the build machine); it is [NOT MEASURED].

11.6 Which lab proves what

Lab	Attacker	Satellite aware of the attacker?	Status	What it proves
W: Windows + Kali VM	explicit relay in a real VM	yes (it targets the relay)	**run**	the full chain across a VM boundary, live
D1: Docker relay	explicit relay	yes	**run**	isolated three-machine run, reproducible in one command
D2: Docker MITM	router + DNAT	**no** (proven by its pcap)	**run and verified**	an invisible attacker still cannot pass a forged product
N: namespaces	router + DNAT	no	run earlier	same, without containers
V: three VMs	Kali router	no	**not booted**	provided for Phase 2

Table 26: Which lab proves what.

12 The ground-station dashboard

> **Rubric: clarity (5) and architecture (15).** We deliver an operator interface where every figure is read live from the API or the result files, with an optional sign-in server. Evidence: `frontend/`, built into `pixelproof/static/app`; `backend/`.

12.1 Pages and data sources

The dashboard is a React 18 + TypeScript application built with Vite into static files that the ground station serves itself, so the jury needs only Python. It works offline (fonts and 3-D scene bundled, no CDN), falls back to a 2-D diagram without WebGL, and enforces a strict Content-Security-Policy. A grouped sidebar (Monitor, Verify, Validate, Docs) becomes a slide-in menu on phones, and a 14-step guided tour opens each page and spotlights the real panel. Nothing on a page is typed by hand: counters come from the audit log, headline figures from the result files.

Page	Question it answers	Data source
Overview	what is the station doing, and how good is it?	`/api/v1/results`, `/stats`, `/ledger`
Live link, Audit log	what arrived, what was decided, is the history intact?	`/api/v1/ledger` (every 3 s)
Station	whom do we trust, and is the system healthy?	`/api/v1/station`
Analyse, Provenance	can I trust this patch, and where did its proof break?	`POST /api/v1/analyse`
Attack lab, Attack guide	does the defence hold against each attack?	`/api/v1/lab/*` (lab mode only)
Results	the measured evidence	`/api/v1/results`

Table 27: Dashboard pages and where each reads its data.

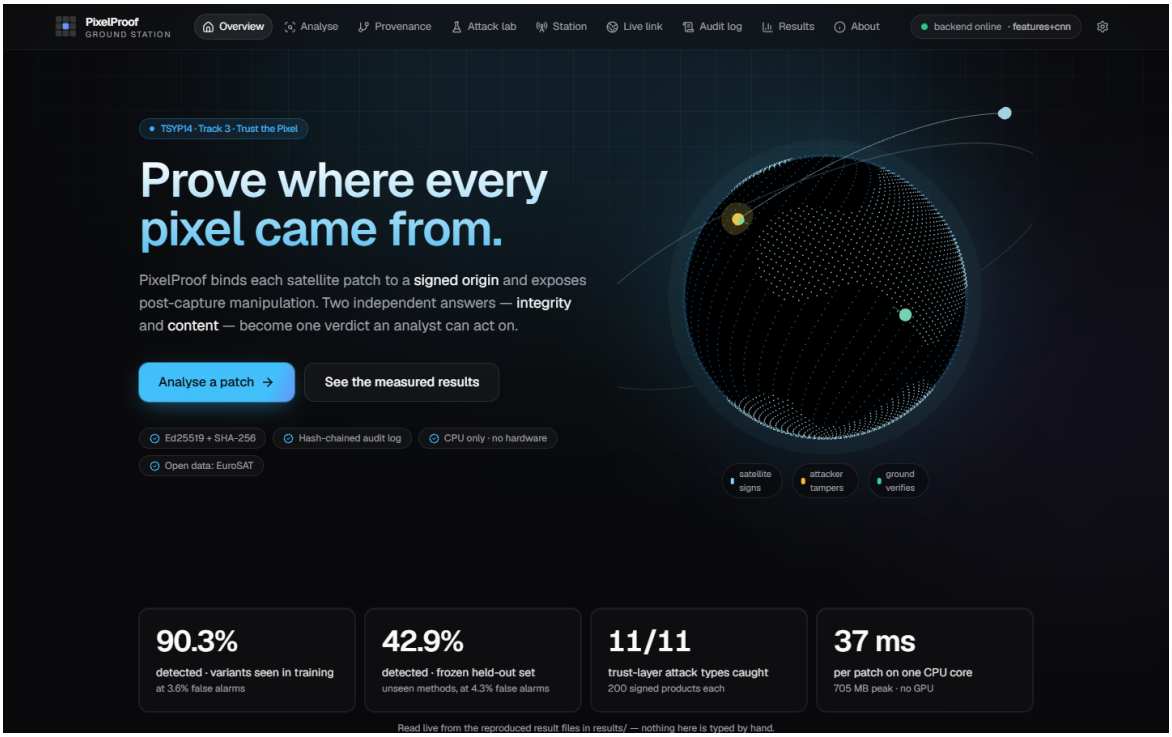


Figure 19: Overview: the headline figures are read from the result files, with the 3-D satellite → attacker → ground globe and the navigation. No number is typed by hand.

12.2 Operator sign-in

A ground station shared by several operators needs an access screen. An optional sign-in server (Laravel 12, `backend/`) provides it; when the server is not running, the dashboard opens in an explicitly labelled *open access* mode (the Overview and Analyse screenshots above were taken in

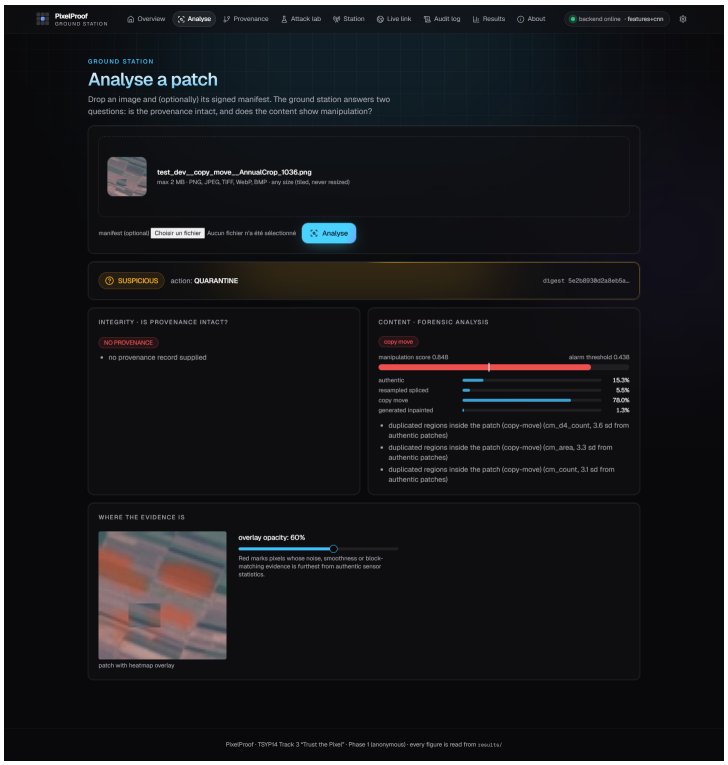


Figure 20: Analyse: a copy-move patch without provenance. Integrity says `NO_PROVENANCE`, the detector says copy-move (score far above the threshold), the verdict is `SUSPICIOUS` and quarantined, and the heatmap marks both copies.

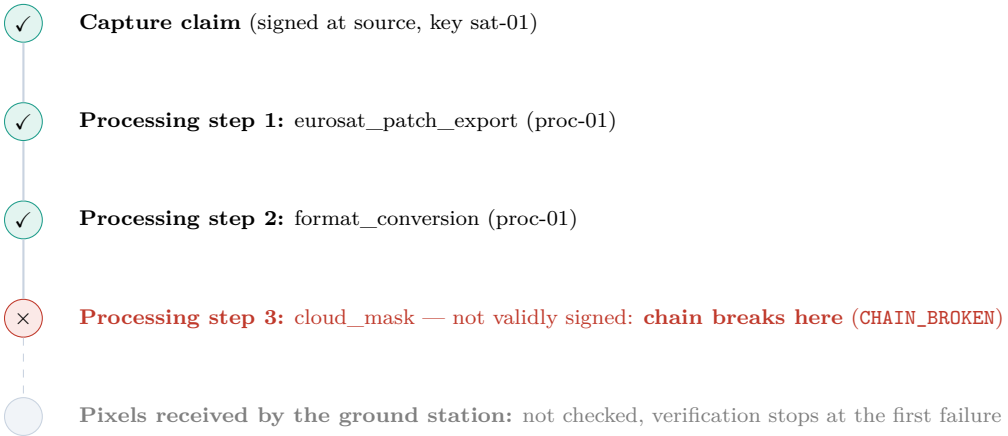


Figure 21: The Provenance page for a forged processing step: the claim and the two genuine steps are green, step 3 is where the chain breaks, and the received pixels are marked "not checked". (Redrawn from the page; the page itself is built from the verifier's reply.)

that mode, so they contain no user name). The sign-in protects the *dashboard*; the ground-station ingest API is unauthenticated in the isolated lab (Section 17).

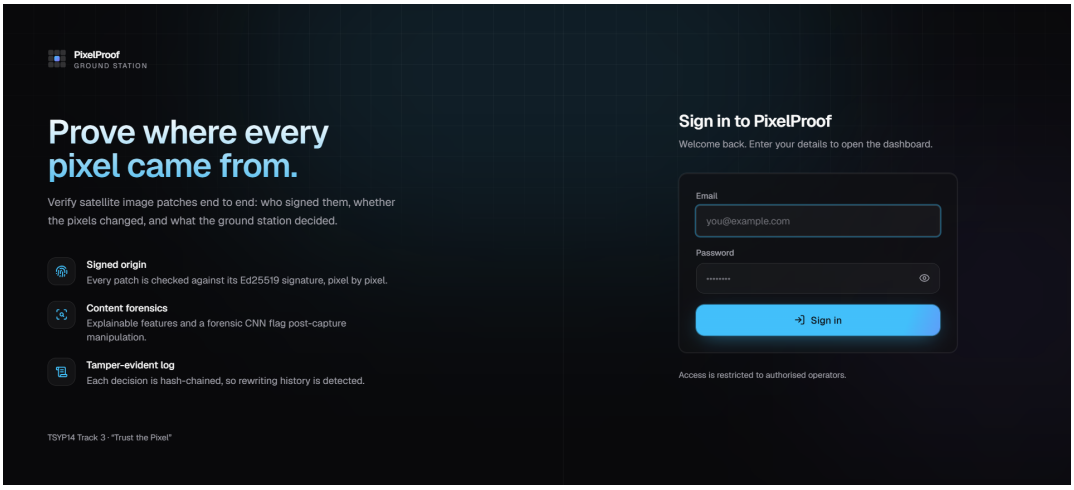


Figure 22: The sign-in screen: e-mail and password only; access is restricted to authorised operators.

Property	How it is provided
Tokens	two JWTs [13] signed HS256: a short-lived **access token** (15 minutes) and a **refresh token** (7 days) carrying a `refresh` claim
Storage in the browser	`HttpOnly` cookies `access_token` and `refresh_token`, `SameSite=Lax`, path `/`; page scripts cannot read them (Figure 24)
Request authentication	a middleware copies the access cookie into the `Authorization: Bearer` header on the server side, so no script handles the token
Refresh and rotation	the refresh token is also stored server-side; using it deletes it and issues a new pair, so a stolen refresh token works once
Logout	the stored refresh token is deleted, the access token is put on the blacklist, and both cookies are expired
Account rules	passwords stored as bcrypt hashes; login requires a verified e-mail address and an active account; login, registration and refresh are rate-limited to 10 requests per minute

Table 28: Properties of the operator sign-in, read from the backend code.

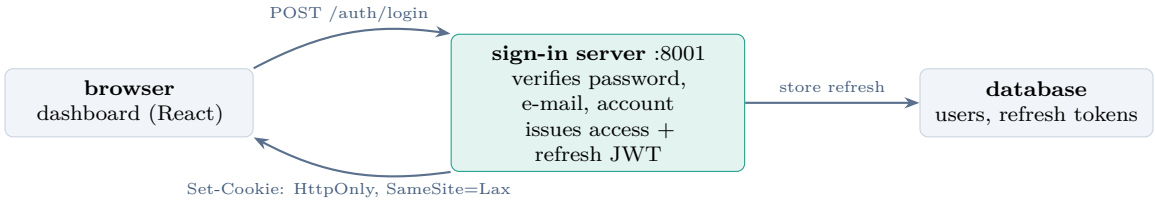
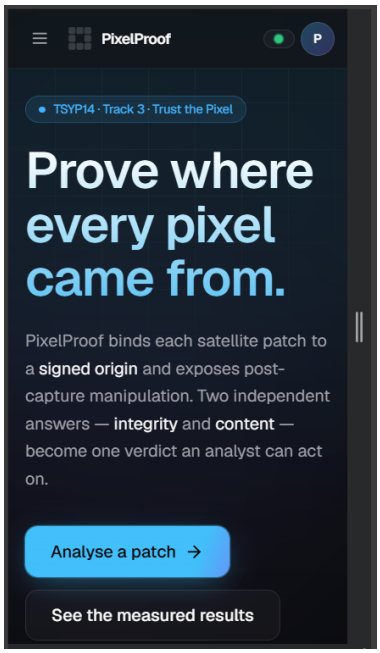


Figure 23: Login flow: the tokens only ever travel in cookies that scripts cannot read; later calls (`/me`, `/auth/refresh`, `/logout`) carry the cookies automatically.

Known limits of this access layer. The cookies are not flagged `Secure` in the current configuration (the lab serves plain HTTP on localhost); a deployment must serve both applications over TLS and set the flag. The sign-in server and the ground station are separate programs, so signing in does not authenticate calls made directly to the ground-station API.

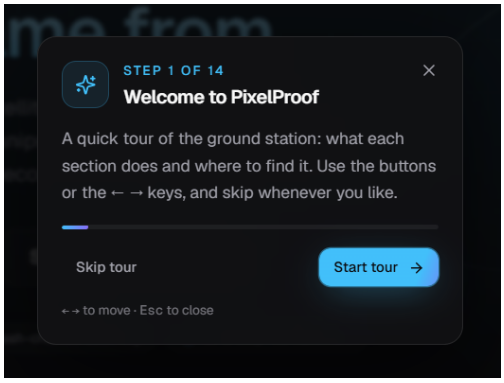


(a) Phone width (320 px).

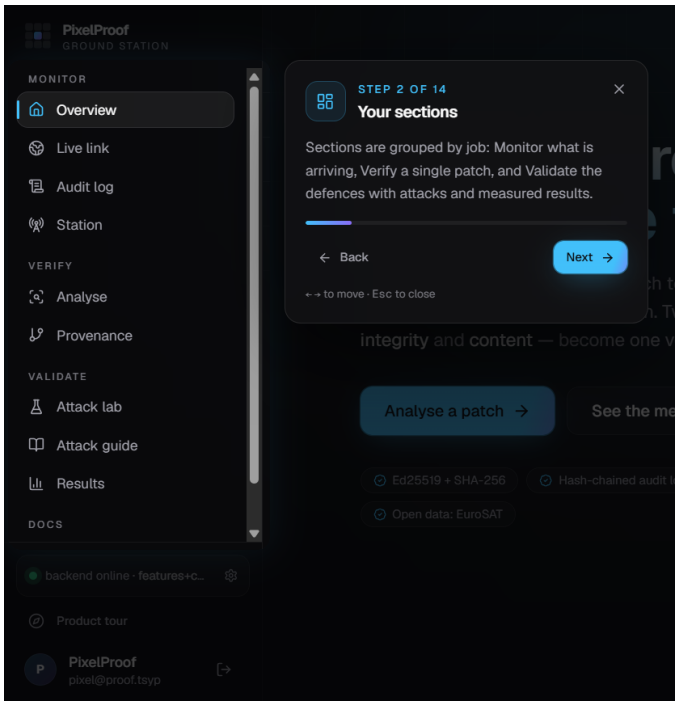
Nom	Valeur	D...	C...	Ex...	Ta...	H...	S...	Sa...	Si...	Cr...	Pr...
_ga	GA1.1.1157986423.179...	lo...	/	2...	30						M...
_ga_9YXXZ3...	GS2.1.s1793651063$o...	lo...	/	2...	59						M...
access_token	eyJ0eXAiOiJKV1QiLCJh...	lo...	/	2...	335	✓		Lax			M...
refresh_token	eyJ0eXAiOiJKV1QiLCJh...	lo...	/	2...	356	✓		Lax			M...

(b) Browser storage after sign-in: the `HttpOnly` column is checked for `access_token` and `refresh_token` (`SameSite` = `Lax`); the two `_ga` cookies are not set by the PixelProof source code. The developer tools are in French.

Figure 24: The responsive layout and the cookie flags, captured in the browser's developer tools.



(a) Step 1 of 14.



(b) Step 2 of 14: the sections, with the signed-in operator shown at the bottom of the sidebar.

Figure 25: The guided product tour: each step opens a page and spotlights its real panel.

13 Feasibility, affordability and local execution

Rubric: feasibility (10). We deliver a CPU-only, laptop-friendly system built on open data and libraries, with no hardware to buy. Evidence: `scripts/latency_probe.py`, `make evaluate`.

Measure (one CPU core, no GPU)	Value
Feature extraction per patch (p50 / p95)	21.4 ms / 29.8 ms
Full analysis per patch (features + CNN + fusion)	36.7 ms (27.3 patches/s); 41.5 ms with heatmap
Peak memory of the analysis process (including PyTorch)	705 MB
Model size (feature model, CNN)	3601 KB + 304 KB
Signing / verification (median)	0.25 ms / 0.38 ms
Signed manifest size (one processing record)	1091 bytes, 17.2% of the PNG

Table 29: Cost of one decision: the challenge asks for latency, memory and security overhead.

The trust layer adds nothing measurable next to the content analysis: signing and verification together take a small fraction of one millisecond, against tens of milliseconds for the forensic features and the CNN.

No GPU, cloud, hardware or paid tool is needed. Without PyTorch the system runs with the feature model alone (its own calibrated threshold), so it works on any machine with Python; the dashboard's Station page and `/api/v1/health` show which mode is active. The Docker labs fit on an ordinary 8 GB laptop, and each VM of the three-VM lab fits in 1.5–2 GB of RAM. The software trust anchor and the emulated satellite are explicitly allowed by the challenge rules; the same signer interface can later sit in front of SoftHSM2 or a TPM without code changes.

14 Business model and potential partners

In one sentence. PixelProof is an open, laptop-sized verification layer that sits between a source of Earth-observation imagery and the people who act on it; the people who need it first are the operators who deliver imagery and the analysts who cannot afford to act on a fake. Everything in this section is a **hypothesis to validate with partners**: no agreement, pilot or customer exists yet, and no price or market size is claimed.

14.1 Why the need is real in Tunisia

Tunisia is building up its own Earth-observation capacity: its first governmental satellite, described by the national mapping and remote-sensing centre (CNCT) as a 6U CubeSat designed for Earth observation, is reported to be in development [16], and a space association and regional remote-sensing bodies are already active around it [15]. Every new source of imagery is also a new place where provenance can be lost. Adding signed origin to a new ground segment from day one costs far less than retrofitting it after products are already in circulation, and the downstream decisions named by the challenge (agriculture and insurance, water management, disaster response) are exactly the ones where a silently altered patch does harm.

14.2 Business Model Canvas

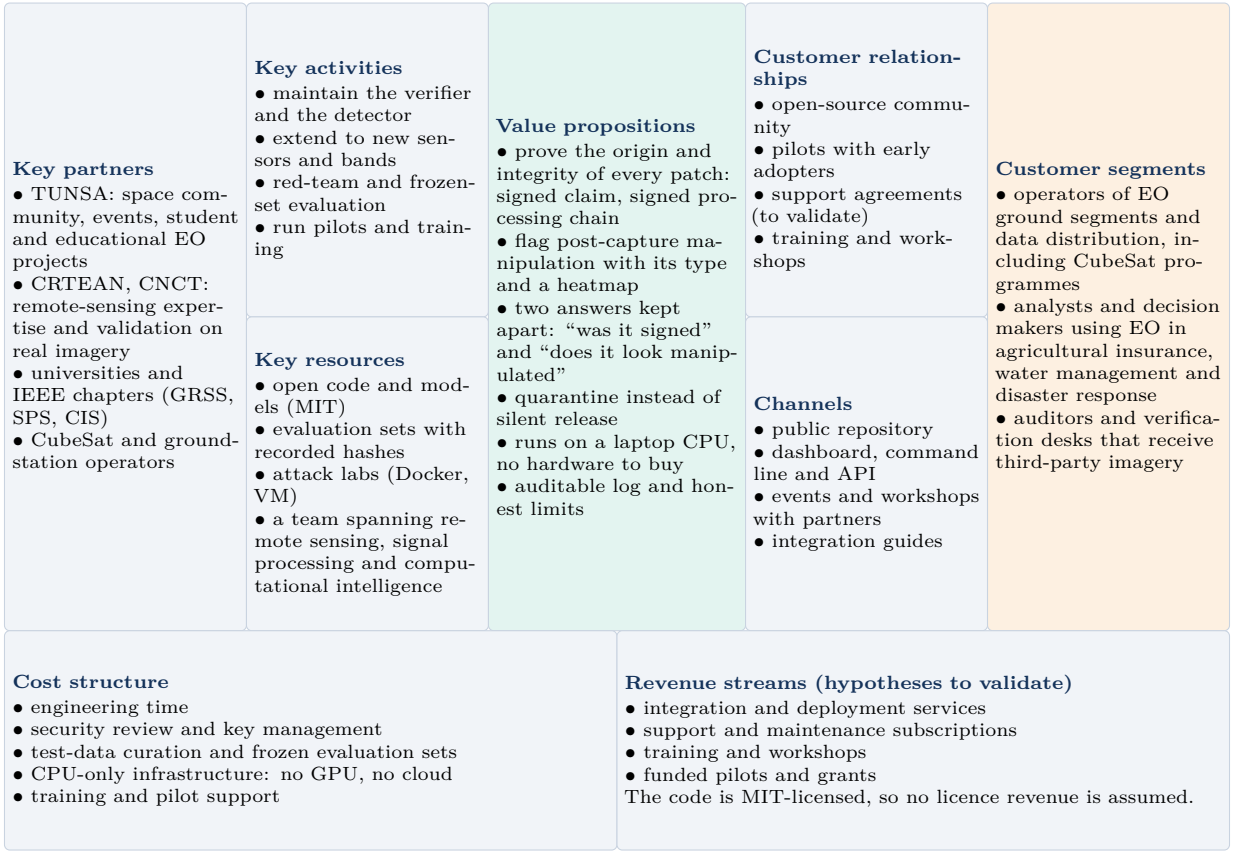


Figure 26: Business Model Canvas of PixelProof. All entries are hypotheses to validate with partners.

Why this model fits the technology. The cost structure is small because the system is CPU-only and open (Section 13); the value proposition is the one thing a signature alone cannot give (Section 7), a verdict that keeps proof and content assessment separate; and the revenue hypotheses rest on services around an open core, which matches how a verification layer is adopted: it has to be inspectable before anyone trusts it.

14.3 Potential partners

None of the organisations below has agreed to anything; the table says what each could bring, what PixelProof would bring in return, and what we would ask for first.

Table 30: Potential partners. Status of every row: not yet approached.

Potential partner	What they could bring	What PixelProof brings	First step
TUNSA, Tunisian Space Association	a non-profit association for space technology in Tunisia; community, events, student and educational EO projects, and an existing cooperation agreement with CRTEAN [15]	a verification layer for the products of educational and student missions; training for their members	present the prototype; offer a pilot on the products of one of their projects

Potential partner	What they could bring	What PixelProof brings	First step
CRTEAN, Regional Center for Remote Sensing of North African States	remote-sensing expertise; access to real satellite imagery for validation [15]	a provenance and integrity check for imagery it distributes or trains on	request a small set of real products to test the detector beyond EuroSAT
CNCT, national mapping and remote-sensing centre	national centre for mapping and remote sensing, technical secretariat of the National Commission of Outer Space Affairs, with remote-sensing studies on water and disaster management [16]	signing at the source and a ground-side verifier designed into national EO products from the start	discuss where a signing point and a registry would sit in a ground segment
Agricultural insurers and agri-risk agencies	the decision context where a doctored patch has a financial cost	flagging and quarantine before imagery enters a claim	ask what false-alarm rate an analyst desk can tolerate
Water-resource and agriculture authorities	operational use of EO for monitoring	provenance and an audit trail	pilot on one monitoring product
Civil-protection and disaster-response agencies	time-critical use of EO where a fake is dangerous	fast, local verification with no cloud dependency	test on a rehearsal scenario
Universities and IEEE chapters (GRSS, SPS, CIS)	independent evaluation, new datasets, student contributors	a documented, reproducible testbed and frozen held-out sets	share the evaluation protocol for independent scoring
CubeSat and ground-station operators	the places where signing keys and processing steps actually live	a software trust anchor with an upgrade path to SoftHSM2 or a TPM	agree a key-handling and registry procedure

14.4 Partnership roadmap

1. **Pilot on a small, friendly source.** Sign the products of one educational or student mission and run the ground-side verifier and dashboard on them (the labs of Section 11 are the rehearsal).
2. **Validate on real imagery.** With a remote-sensing body, score the detector on real Sentinel-2 or CubeSat products and re-measure the false-alarm budget, because EuroSAT is single-JPEG and 8-bit (Section 17).
3. **Move toward users.** Take the quarantine and audit workflow to an insurer, a water authority or a disaster-response desk, and let their tolerance for false alarms set the threshold.

What we would need from a partner: a handful of real signed or signable products, a place to host a registry of public keys, and honest feedback on how many false alarms a human reviewer will accept.

15 Testing, documentation and reproducibility

Rubric: clarity & reproducibility (5). We deliver four test layers, safeguards that make silent errors visible, run and data guides, seeds, a model card, a Makefile and a number-check script. Evidence: `tests/`, `frontend/tests/`, `docs/TESTING.md`, `README.md`, `docs/RUN_GUIDE.md`, `make reproduce`.

15.1 Testing and quality assurance

Layer	Command	What it covers
Python (unit, integration, security, attack)	`python -m pytest -q`	every trust-layer attack, replay after a restart, the scenario engine, features and generators, jury input formats, the detector contract, the API (rate limit, size limit, no stack traces, no key material, lab off by default), the training guard, and the candidate safeguards
Dashboard unit	`npx vitest run`	verdict rendering, upload validation, WebGL fallback, results summary
Dashboard end-to-end	`npx playwright test`	every page opens, the analyser asks both questions, the attack guide renders
Report numbers	`python scripts/check_numbers.py docs/REPORT.md`, `python scripts/render_latex.py --check`	every number in the reports resolves to a result file

Table 31: The four test layers.

Safeguards. Training refuses to overwrite the shipped model unless explicitly asked; the Station page shows the model file's real SHA-256 and whether it matches its model card; the held-out sets are hash-checked before scoring, and held-out v3 refuses to score the same model twice; a test checks that the shipped detector, run through the current code, still reproduces the probabilities stored in `results/predictions.csv.gz`; and `scripts/anonymity_check.py` searches the submission for identifying words, e-mails, home paths and image or PDF metadata.

15.2 Documentation, data and reproducibility

How the numbers in this report are produced. Every measured number is inserted from the result files in `results/` by `scripts/render_latex.py`, which refuses to build the PDF when a number cannot be resolved. The only exceptions are the live Windows + Kali session (Section 11.2) and the Phase 2 candidate (Section 16), whose values are typed in the source and say so where they appear. Values are labelled [MEASURED], [TARGET] or [NOT MEASURED]. We never write "perfect" or "unbreakable", and never call a patch "authentic", only "no traces found".

- `README.md` explains the whole system and every command; `docs/RUN_GUIDE.md` is the jury's clean-machine guide; `vm/README.md` is the step-by-step lab runbook.
- `scripts/download_eurosat.py` downloads the official release and refuses data whose class counts differ; the copy used for our numbers has a published content fingerprint (`docs/DATASET_CARD.md`).
- All randomness is seeded; every evaluation patch is rebuilt deterministically from (patch id, split, family). Dependency versions are pinned (`requirements.txt`, `requirements-cnn.txt`).

- The model card (`models/detector.json`) records the model's thresholds, its fusion weight and its SHA-256:

`927f0930a148c56281c27362f6922797a6586b4ef848a51078f020f115e1e198`

make target	What it runs
`make test`	`python -m pytest -q`
`make demo`	`python scripts/quick_demo.py` (ground + attacker + satellite)
`make quick`	a 5-minute end-to-end check on one fifth of the data
`make reproduce`	features → training → attack campaign → evaluation
`make campaign`, `make evaluate`, `make heldout`	each step on its own
`make check`, `make report`	number check, then the report PDF
`make anonymity`, `make delivery`	anonymity search, submission zip

Table 32: Make targets. `make` does not exist on Windows; every target is one plain `python ...` line that can be run by hand.

15.3 Phase 1 deliverables and Phase 2 next steps

	Phase 1 (this submission)	Phase 2 (next steps)
Trust	signed claim and chain, key registry, algorithm pinning, replay memory rebuilt from the log, hash-chained audit log, 11-attack campaign, scenarios A–F	authentication and TLS on the ingest API, an external anchor for the audit log, processor key kept on the ground system
Detection	48 features, GBM + PixelNet fusion, re-encoding gate, calibrated budget, heatmap and evidence text	calibrated "globally processed / inconclusive" output, adoption of the candidate detector once its re-encoding cost is fixed, RANSAC grouping for copy-move
Validation	seen and held-out evaluation, frozen v2, robustness sweep, ablation, bootstrap intervals, four lab architectures	training and testing on public GAN or diffusion satellite fakes; multispectral inter-band consistency
Operator	dashboard (ten pages and a guided tour), optional sign-in	the live demonstration on the Windows + Kali lab, with the transparent variant on Docker
Documentation	this report, run guide, data card, threat model, security analysis, testing and reproducibility notes	final slides and a replayable attack demonstration for the pitch

Table 33: Phase 1 deliverables and Phase 2 next steps.

16 Phase 2 work in progress: a candidate detector, measured

Not shipped, and typed values. Every number in the other sections belongs to the shipped detector. This section reports a retrained candidate measured with the same tools, so the jury can see the direction of Phase 2 and its cost. The candidate's evaluation figures below are typed in this document; the repository snapshot this document is built from

holds the candidate model, its training log and its predictions (`models_candidate_w16/`, `results/candidate/w16/`) but no regenerated metrics file, so unlike every other number they are not produced by `render_latex.py`.

16.1 Why, and what changed

The shipped CNN was still improving when its training stopped (validation accuracy rising at the last of its 8 epochs), and its weakest families are unseen splicing and copy-move. The candidate keeps the architecture and the features and changes only the training:

Change	Shipped	Candidate
Training edits	`dev` variants	`dev` + `dev_plus`: irregular blob-shaped edits absent from every test set (nearest/shear resampling, blob splices with brightness matching, blob copy-move with a gain change, blob fills)
CNN epochs and selection	8 epochs, best validation accuracy	up to 24 epochs, early stopping on validation recall at the 5% operating point
Regularisation	none	dropout 0.1, label smoothing 0.05
Test-time averaging	no	average over the 8 flips and rotations
Fusion weight (features)	0.4	0.1 (chosen on validation)

Table 34: What the candidate changes. Every choice was made on validation data. A wider network (width 32) was planned but not trained.

16.2 Held-out v3, frozen before the candidate was trained

Held-out v2 had already been used to measure the changes after v1, so it is no longer unbiased for a further change. We therefore built held-out v3 (Section 5) with methods and shapes that appear nowhere else, rendered 6680 patches and recorded their SHA-256 before the candidate was trained. The script refuses to re-freeze it and refuses to score the same model twice.

16.3 Results (fused models)

Test set	Shipped detection	Shipped false alarms	Candidate detection	Candidate false alarms
seen variants (test-seen)	90.3%	3.6%	97.5%	5.0%
held-out v1	48.2%	4.4%	75.2%	4.3%
held-out v2 (biased, see above)	42.9%	4.3%	84.6%	4.3%
held-out v3 (first and only look)	51.1%	3.4%	**98.7%**	5.0%

Table 35: Shipped versus candidate detector on four test sets. The shipped columns of the first three rows come from the result files; the candidate columns and the v3 row are typed.

16.4 The cost, and how to read the gain

- **The gain is large and consistent** across three held-out sets, including v2, which was designed independently of the new training edits.

Family	held-out v1	held-out v2	held-out v3
resample	96.6% → 100.0%	67.5% → 99.8%	97.4% → 100.0%
splice	18.1% → 64.7%	17.4% → 47.7%	34.1% → 94.2%
copy-move	25.3% → 63.9%	41.7% → 81.0%	35.7% → 98.8%
inpaint	63.8% → 80.1%	45.1% → 99.9%	52.5% → 100.0%

Table 36: Detection by family, shipped → candidate.

Robustness (false alarms on authentic patches)	Shipped	Candidate
no operation	3.5%	6.7%
JPEG q90	5.5%	23.7%
JPEG q70	3.8%	3.0%
WebP q60	7.2%	3.7%
analysis time per patch (one core)	36.7 ms	74.7 ms

Table 37: The candidate's price (shipped column from the result files, candidate column typed).

- **The cost is real:** more false alarms on re-encoded authentic patches, and about twice the analysis time (the 8-fold test-time averaging; the candidate was also timed on a busier machine).
- **Caveat on v3.** v3 and the new training edits were written by the same authors; their shapes and methods are disjoint, but v3 may still be closer to them than a truly unknown attack. The jury's held-out patches remain the real test, which is why the shipped model stays the official one until the re-encoding cost is fixed.

Phase 2 plan for the candidate: recalibrate the re-encoding gate for the new score distribution, train the width-32 variant, and adopt the model only if it keeps the false-alarm budget on re-encoded authentics.

17 Assumptions and limitations

17.1 Assumptions and technical risks

17.2 Limitations, with numbers

- **Smoothing of authentic patches** (blur, JPEG 2000) still raises false alarms sharply (Table in Section 10.3): at JPEG 2000 rate 30 the false-alarm rate is 87.3%. We do not yet distinguish benign smoothing from resampling: the patch is quarantined for review, which is a false-alarm cost.
- **Held-out splicing and copy-move remain the hardest families** for the shipped model: on held-out v1 splicing is detected at 18.1% and copy-move at 25.3%; on frozen v2 at 17.4% and 41.7%.
- **Localisation** is weak except copy-move (IoU 0.31; 0.11 for splicing and 0.02 for inpainting).
- **Integrity cannot judge content signed by a compromised source**: in scenario F1 the signature is valid for every fake and the detector flags only 9 of 20.
- **The audit log is tamper-evident, not tamper-proof**, and the ground-station API has no authentication or TLS (isolated lab). The optional sign-in protects the dashboard's access, not the API, and its cookies are not flagged `Secure` on plain HTTP.

Assumption or risk	How we handle it
We design both the manipulations and the detector (circular simulation)	held-out methods never seen in training, frozen sets with recorded SHA-256, a v3 set for later changes; the jury's patches are the real test
A software or emulated trust anchor is acceptable	explicitly allowed by the rules; same signer interface for SoftHSM2 or a TPM; fingerprints compared out of band
EuroSAT RGB is single-JPEG and 8-bit	re-encoding gate and laundering-robust training; multispectral and 16-bit inputs are converted but their false-alarm rate is [NOT MEASURED]
No GAN or diffusion patches in the official data	the "generated" class is approximated by inpainting and texture synthesis, clearly labelled, not a claim about GAN imagery
A compromised source signs a fake	stated as a limit (scenario F1); content analysis still runs on signed data; revocation
Learned detectors degrade on unseen manipulations	provenance carries the guarantee; held-out numbers are reported next to the seen ones
A drifting or poisoned model	training refuses to overwrite the shipped model; model file hash shown on the Station page and checked against the model card
Partners and revenue are hypotheses	the business model and every partner of Section 14 are unvalidated; no agreement, pilot or price exists, and nothing in this report depends on them

Table 38: Assumptions and technical risks.

- **In the lab, the processor key is created next to the satellite key**, and in Lab W the satellite writes the public registry that the ground station reads (emulation shortcuts). A real deployment keeps the processor key on the ground system.
- **Provenance-stripped edits are only as good as the detector:** of 200 edited patches with the manifest removed, 62 were released as `UNVERIFIED_NO_TRACES`.
- **The frozen held-out v2 set has now been used to measure the effect of our changes** (Section 10). It is a fair before/after comparison, but it is no longer an unbiased estimate for any further change.
- **Lab sizes are small** (tens of messages per Docker run, 20 trials per scenario); the 200-product campaign is the more meaningful trust-layer result.
- **The three-VM VirtualBox lab was not booted by us**; the Windows + Kali VM lab, the Docker labs and the namespace lab were run.

Abbreviations

Term	Meaning
Earth observation and imagery	
EO	Earth observation
Sentinel-2	the Copernicus optical satellite pair whose patches EuroSAT is built from
EuroSAT	the public dataset of 27,000 labelled Sentinel-2 patches (64×64, 10 land-cover classes)
Patch / tile	a 64×64 image; larger images are cut into tiles, never resized
Resampling	rescaling or rotating an image by interpolation; leaves periodic pixel correlations

Term	Meaning
Splicing	pasting a region from another image
Copy-move	duplicating a region inside the same image
Inpainting	erasing a region and filling it from its surroundings (classical or learned)
Laundering	post-processing (re-compression, blur, noise) that hides forensic traces
Security and provenance	
Digest	a SHA-256 fingerprint of the pixels; any pixel change alters it
Ed25519	an elliptic-curve signature scheme with deterministic signatures
Claim / manifest	the signed statement of origin: source, sensor, capture date, digest
Processing chain	the signed, linked record of every processing step
Registry	the ground station's list of trusted public keys, with roles and revocation flags
Replay	re-sending a genuine, already delivered message
MITM	man-in-the-middle: an attacker on the path between sender and receiver
DNAT	destination network-address translation (the iptables rule of Lab D2)
PKCS#11, HSM, TPM	interfaces and devices for hardware-backed keys (upgrade path of the trust anchor)
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
JWT	JSON Web Token (the sign-in tokens)
TLS	transport-layer security (not enabled in the isolated lab)
LSB	least-significant bit (hidden-data embedding)
Detection and evaluation	
GBM	gradient-boosted trees (`HistGradientBoostingClassifier`)
CNN	convolutional neural network (PixelNet)
SRM	spatial rich model: fixed high-pass kernels that expose sensor noise
TLU / truncation	clipping the residual to ± 3
DCT	discrete cosine transform (block matching for copy-move)
D4	the eight rotations and reflections of a square
MAD	median absolute deviation (robust scale)
AUC-ROC, AUC-PR	area under the ROC or precision–recall curve
Detection rate	fraction of manipulated patches flagged
False-alarm rate	fraction of authentic patches wrongly flagged
Macro-F1	F1 averaged over the four classes
IoU	intersection over union (heatmap versus true mask)
Seen / held-out	manipulation methods present in / absent from training
Frozen set	an evaluation set whose SHA-256 was recorded before the change it is used to measure
Re-encoding gate	a model estimating whether a patch was re-compressed, used to move the alarm threshold
Budget	the target false-alarm rate on clean validation authentics
Organisations and challenge	
TSYP14	the 14th IEEE Tunisian Student and Young Professional Congress
BMC	Business Model Canvas
TUNSA	Tunisian Space Association
CRTEAN	Regional Center for Remote Sensing of North African States

Term	Meaning
CNCT	Centre national de la cartographie et de la télédétection (Tunisian national mapping and remote-sensing centre)
GRSS, SPS, CIS	the IEEE Geoscience and Remote Sensing, Signal Processing and Computational Intelligence Societies

A How to run it

```
pip install -r requirements.txt            # CPU only
pip install -r requirements-cnn.txt --index-url https://download.pytorch.org/whl/cpu   # the CNN
python -m pytest -q                        # automated tests
python scripts/quick_demo.py               # ground + attacker + satellite; dashboard at :8000
python -m pixelproof.cli detect --input <jury_dir> --output predictions.csv   # held-out patches
python scripts/download_eurosat.py && make reproduce TRAIN_FLAGS=--overwrite  # every number
python scripts/render_latex.py             # this report (needs pdflatex)
```

B Integrity statuses and verdicts

Integrity status	Meaning
`VERIFIED`	signed by a trusted source, chain intact, pixels match, date plausible, not a replay
`NO_PROVENANCE`	no manifest: the content detector decides
`MALFORMED`	missing or invalid manifest fields
`WEAK_ALGORITHM`	an algorithm other than sha256 + ed25519 is claimed
`UNKNOWN_KEY` / `REVOKED_KEY`	the signer is not trusted / was revoked
`BAD_SIGNATURE`	the claim was modified after signing
`CHAIN_BROKEN`	a processing step is unsigned, unlinked or inconsistent
`PIXELS_ALTERED`	the pixels differ from the last signed digest
`TIME_IMPLAUSIBLE`	capture time in the future or before the sensor existed
`REPLAY`	product already received, or sequence number not increasing

Table 40: Integrity statuses.

Verdict	Action
`VERIFIED`, `UNVERIFIED_NO_TRACES`	release
`SIGNED_BUT_SUSPICIOUS`, `ALTERED_AFTER_CAPTURE`, `SUSPICIOUS`	quarantine (kept, withheld, released only by a logged analyst decision)

Table 41: Verdicts and the response to each.

C API reference

The optional sign-in server exposes `POST /api/auth/login`, `/register`, `/refresh`, `/logout` and `GET /api/me` (Section 12.2).

D Repository map

Method and path	Purpose
`GET /api/v1/health`	status, model version, detector mode (`features+cnn` or `features`), number of trusted keys
`POST /api/v1/ingest`	a live downlink (`image_png_b64`, `manifest`): verify (and record), analyse, decide, log
`POST /api/v1/analyse`	an analyst upload (multipart image + optional manifest): inspect only, no replay record
`GET /api/v1/ledger?limit=`	recent audit-log entries, chain validity and first bad entry
`GET /api/v1/stats`	counters computed from the audit log only
`GET /api/v1/station`	trusted public keys and fingerprints, trust policy, replay memory, detector mode and model hash
`GET /api/v1/registry`	the public-key registry
`GET /api/v1/results`	the measured result files (for the dashboard)
`GET /api/v1/lab/scenarios`, `POST /api/v1/lab/run`, `GET /api/v1/lab/runs`	attack scenarios (only with `PIXELPROOF_LAB_MODE=1`)

Table 42: Ground-station API. Limits: 2 MB per image, decode guard, per-client rate limit on the analysis endpoints (`PIXELPROOF_RATE_PER_MIN`, default 120 per minute), no stack traces or key material in responses (tested).

Path	Role
`pixelproof/`	ground station: trust layer, detector, API, attack engine, CLI (Section 6)
`pixelproof/static/app/`	built dashboard served by the ground station
`frontend/`	dashboard source (React, TypeScript); `npm run build`
`backend/`	optional Laravel sign-in server (source only; `composer install`)
`simulation/`	`satellite_node.py`, `attacker.py`, `score_demo.py`, `run_attack.py`
`scripts/`	data, training, evaluation, held-out sets, campaign, demo, report, delivery, anonymity
`models/`, `models_v1/`	shipped detector (+ model card) and the v1 model behind the before/after evidence
`models_candidate_w16/`, `results/candidate/`	the not-shipped candidate detector and its logs (Section 16)
`results/`	every measured number and the lab evidence (`attack_runs/`, `docker_lab/`, `docker_mitm/`, `lab/`)
`docker-compose.yml`, `docker-compose.mitm.yml`, `Dockerfile`	Docker labs D1 and D2
`lab/`, `vm/`	VirtualBox lab (Vagrantfile, provisioning) and the lab runbook
`samples/`	30 unmodified EuroSAT patches (labs and tests run without the dataset)
`tests/`, `frontend/tests/`	automated tests
`docs/`	report sources (Markdown and LaTeX), threat model, security analysis, attack scenarios, data card, guides

Table 43: Repository map.

E Data card (summary)

EuroSAT RGB (Sentinel-2, 27,000 patches, 64×64, 10 land-cover classes), MIT-licensed, obtained through the official release. Splits by SHA-256 of the patch id with donor and manipulation co-assignment (no leakage). 6680 seen-test and 6612 held-out-test patches; held-out v3 holds 6680 patches. Full details, manipulation parameters and the content fingerprint: `docs/DATASET_CARD.md`.

References

[1] P. Helber et al. EuroSAT: a novel dataset and deep learning benchmark for land use and land cover classification. *IEEE JSTARS*, 2019.

[2] A. C. Popescu and H. Farid. Exposing digital forgeries by detecting traces of resampling. *IEEE Transactions on Signal Processing*, 2005.

[3] J. Fridrich, D. Soukal and J. Lukáš. Detection of copy-move forgery in digital images. *DFRWS*, 2003.

[4] J. Fridrich and J. Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 2012.

[5] B. Bayar and M. C. Stamm. Constrained convolutional neural networks for image manipulation detection. *IEEE Transactions on Information Forensics and Security*, 2018.

[6] J. Horváth et al. Manipulation detection in satellite images using vision transformers. *CVPR Workshops*, 2021.

[7] B. Zhao et al. Deep fake geography? When geospatial data encounter artificial intelligence. *Cartography and Geographic Information Science*, 2021.

[8] D. J. Bernstein et al. High-speed high-security signatures (Ed25519). *Journal of Cryptographic Engineering*, 2012.

[9] P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. *ACM Transactions on Graphics*, 1983.

[10] C2PA. Content Credentials technical specification, 2024.

[11] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, 2017.

[12] National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS PUB 180-4, 2015.

[13] M. Jones, J. Bradley and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.

[14] A. Shostack. *Threat Modeling: Designing for Security*. Wiley, 2014.

[15] Space in Africa. TUNSA to advance partnership with CRTEAN to boost collaborative efforts, 8 February 2024. `https://spaceinafrica.com/2024/02/08/tunsa-to-advance-partnership-with-crtean-to-boost-collaborative-efforts/`

[16] Space in Africa. Tunisia moves to develop its first governmental satellite, 19 May 2026. `https://spaceinafrica.com/2026/05/19/tunisia-moves-to-develop-its-first-governmental-satellite/`